Tutoría 1. Lenguajes, gramáticas, autómatas y lenguajes regulares

Elementos esenciales de lenguajes de programación (ISIS 2111). Monitoría

Plan de hoy

Minutos	Parte	Clase del curso
0 a 15	1. Qué es un lenguaje y qué es un programa	Programming languages
15 a 35	2. Gramáticas	Creating (programming) languages
35 a 60	3. Autómatas: DFA y NFA	Automata theory
60 a 85	4. Lenguajes regulares	Regular languages
85 a 90	Cierre y ticket de salida	

Parte 1

Qué es un lenguaje y qué es un programa

¿Qué es un lenguaje de programación?

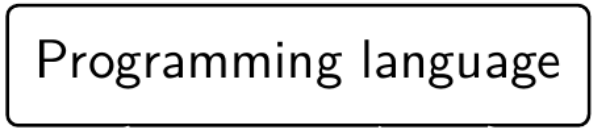
A programming language is a system to write computer programs

¿Qué es un lenguaje de programación?

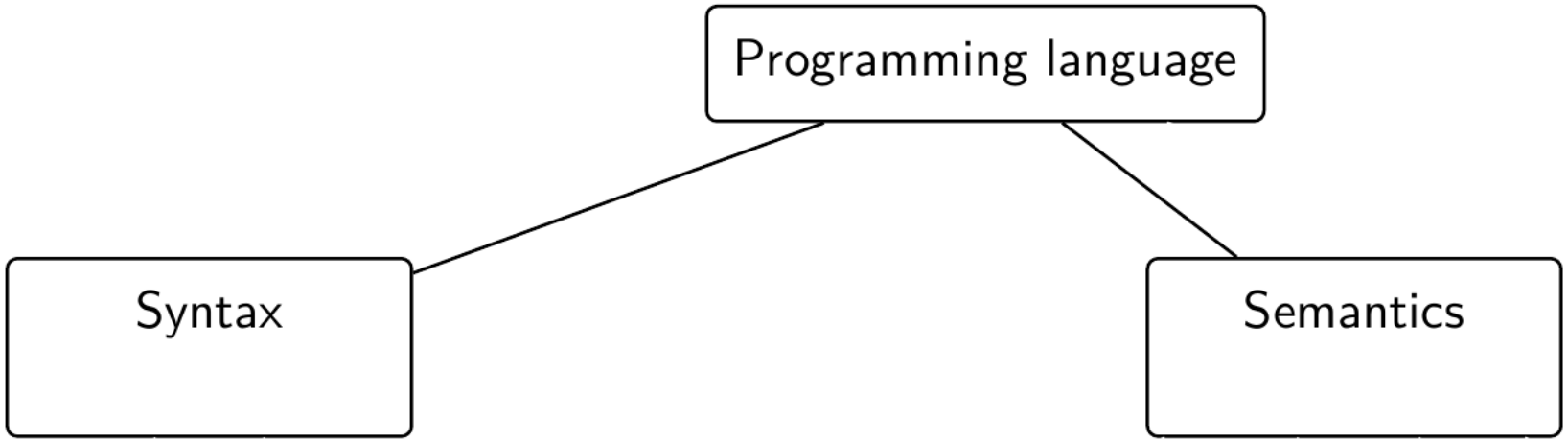
A programming language is a system to write computer programs

¿Qué le falta a esta definición para que sirva de algo?

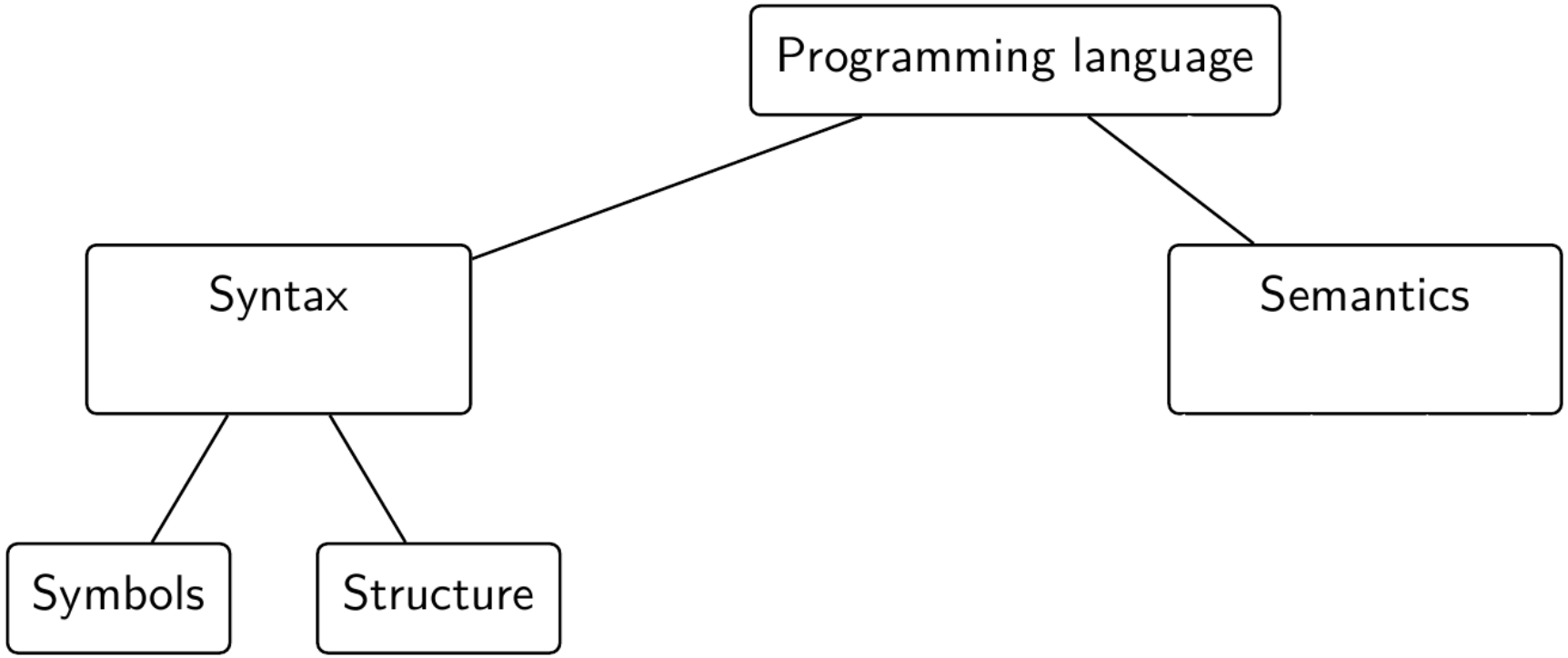
Las partes de un lenguaje



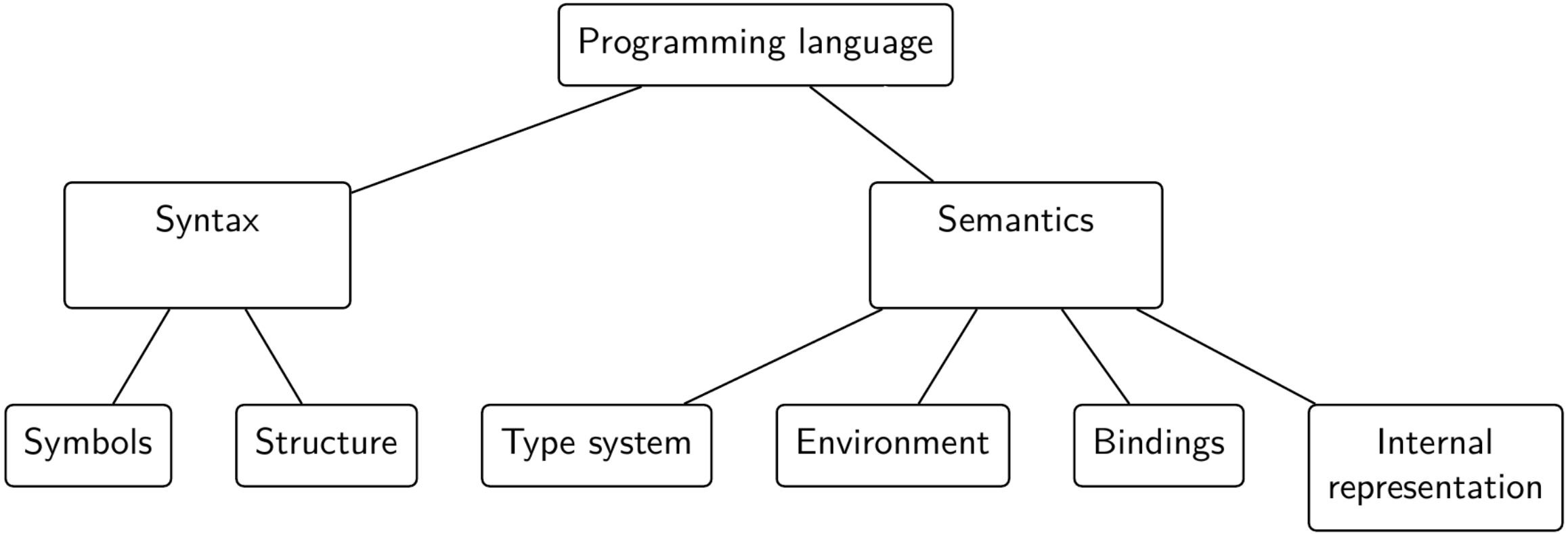
Las partes de un lenguaje



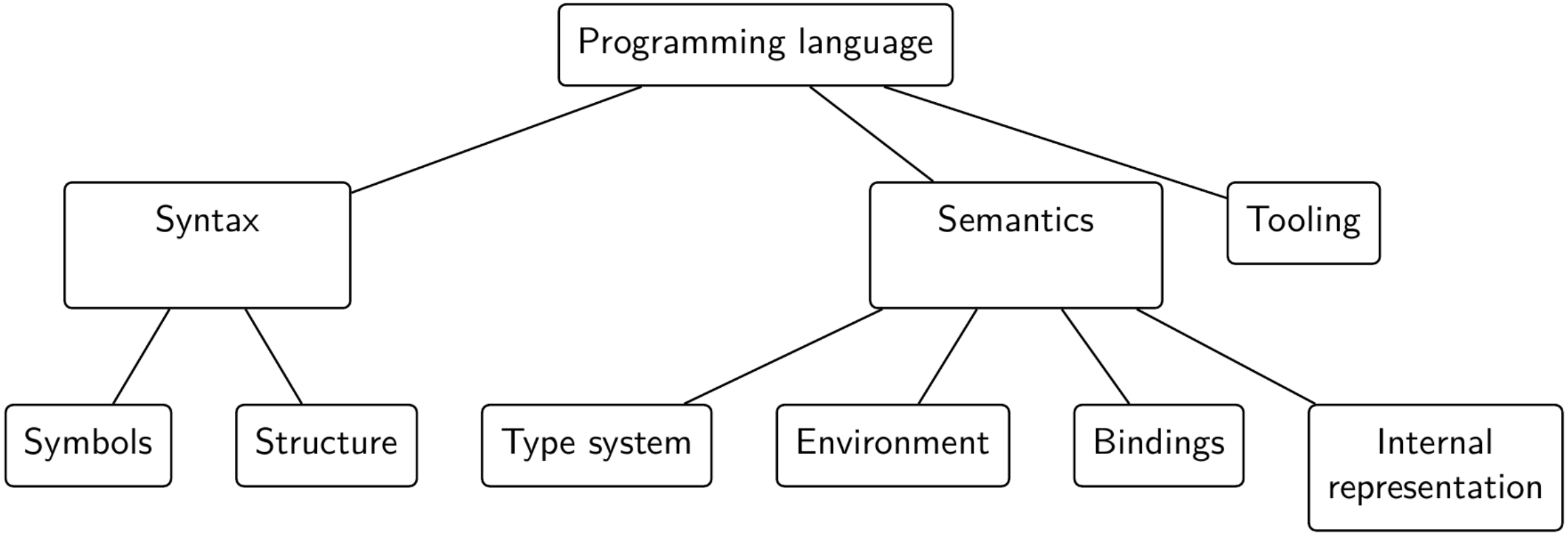
Las partes de un lenguaje



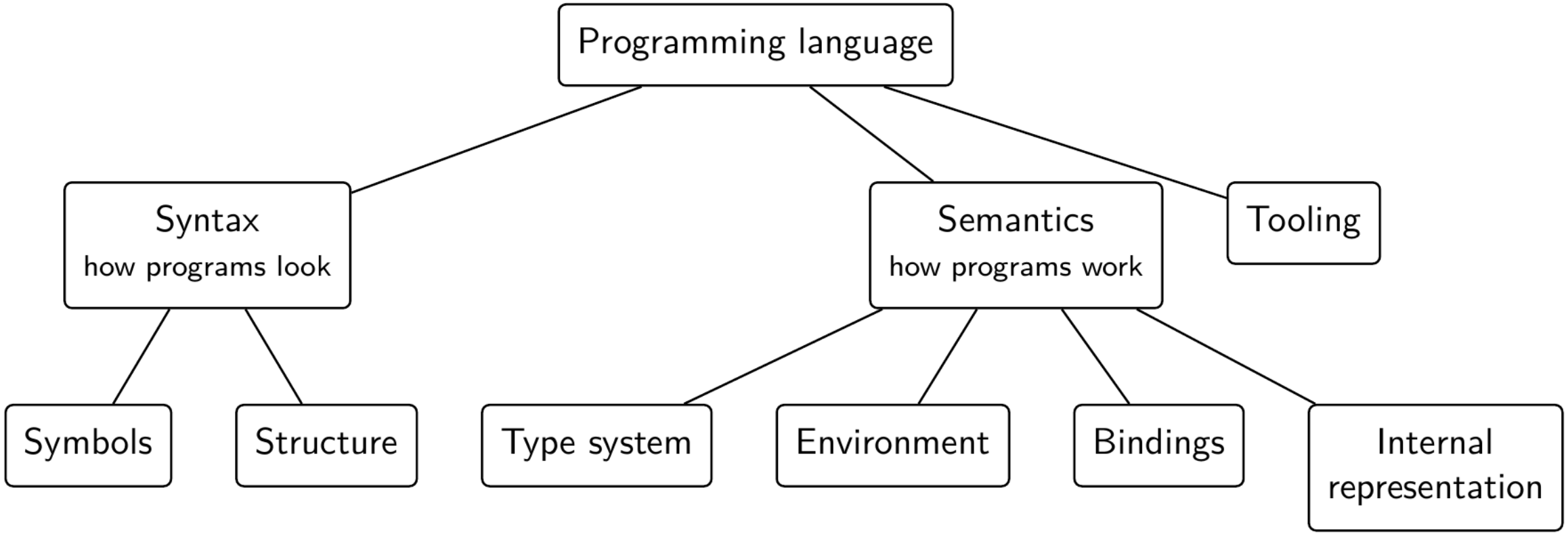
Las partes de un lenguaje



Las partes de un lenguaje



Las partes de un lenguaje



¿Es esto un programa?

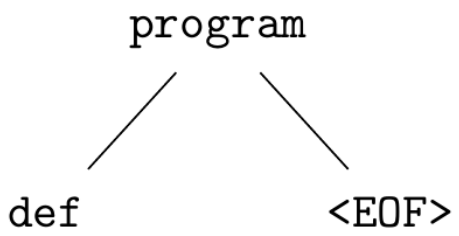
```
fun arithmetic_logic(x,y) {
  a = 3+foo;
  x and y;
}
```

¿Es esto un programa?

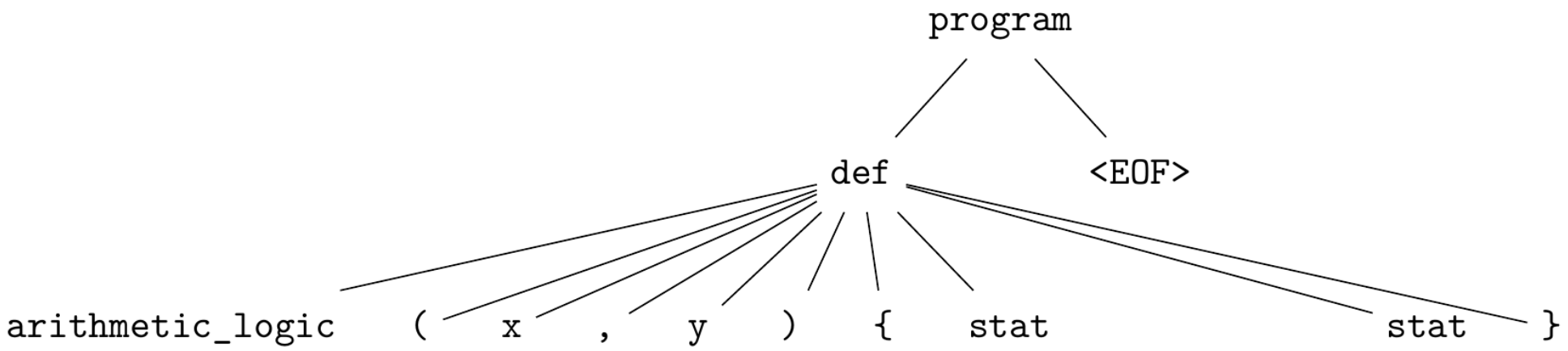
```
fun arithmetic_logic(x,y) {
  a = 3+foo;
  x and y;
}
```

- Sí
- No
- No se puede saber

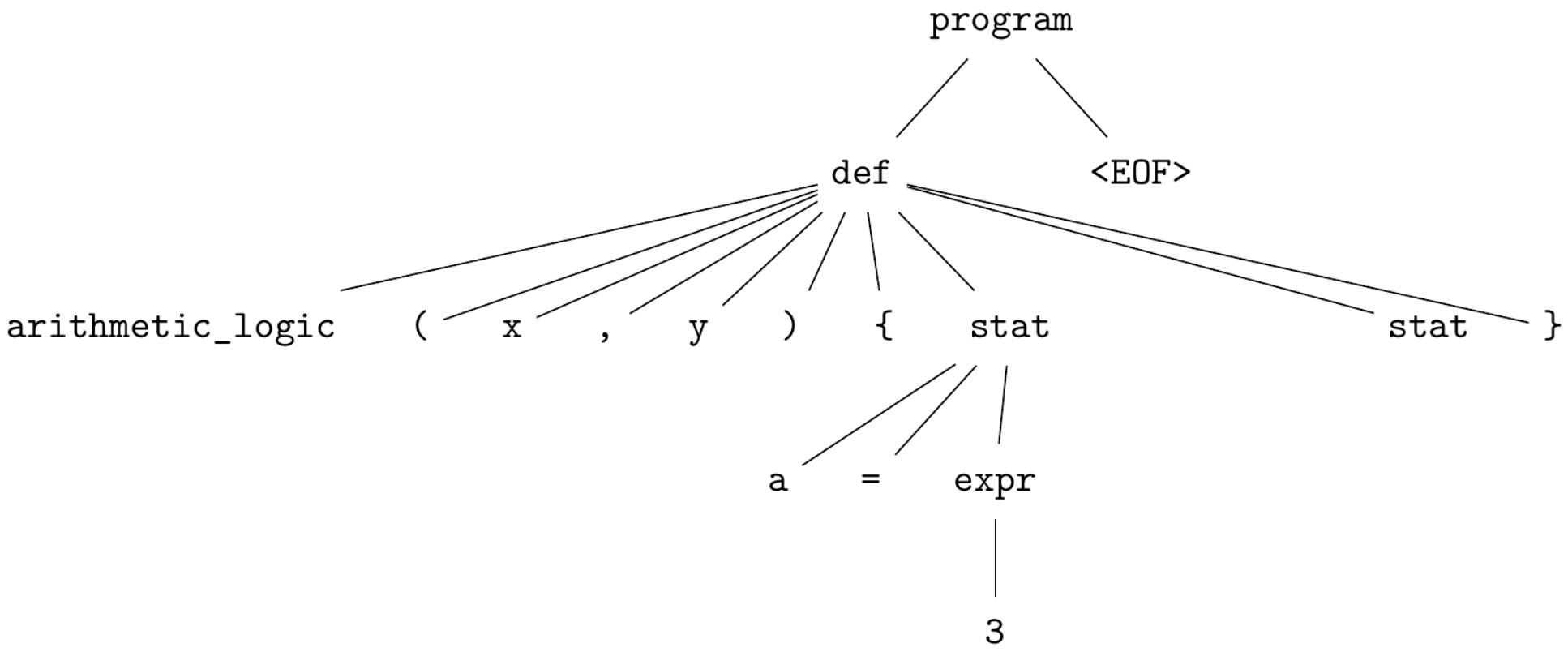
Lo que hace el parser de un lenguaje pequeño



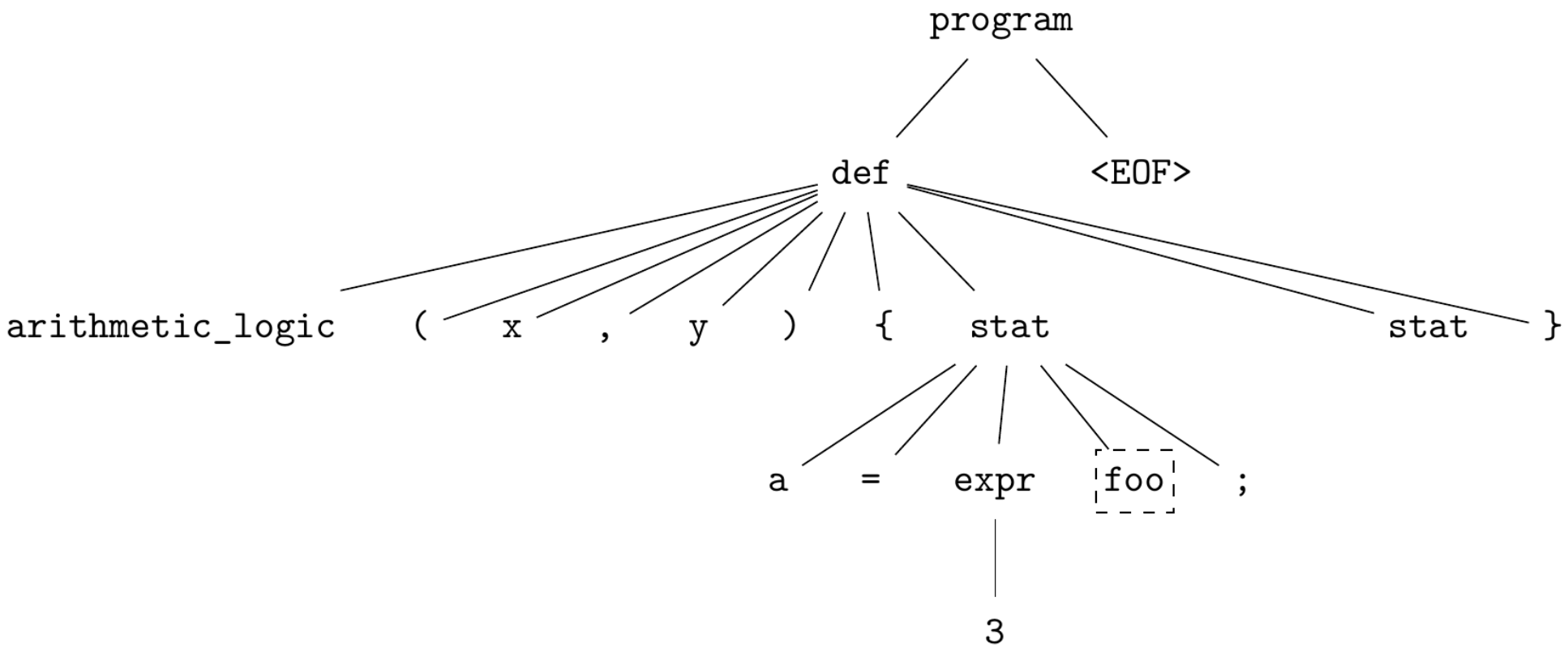
Lo que hace el parser de un lenguaje pequeño



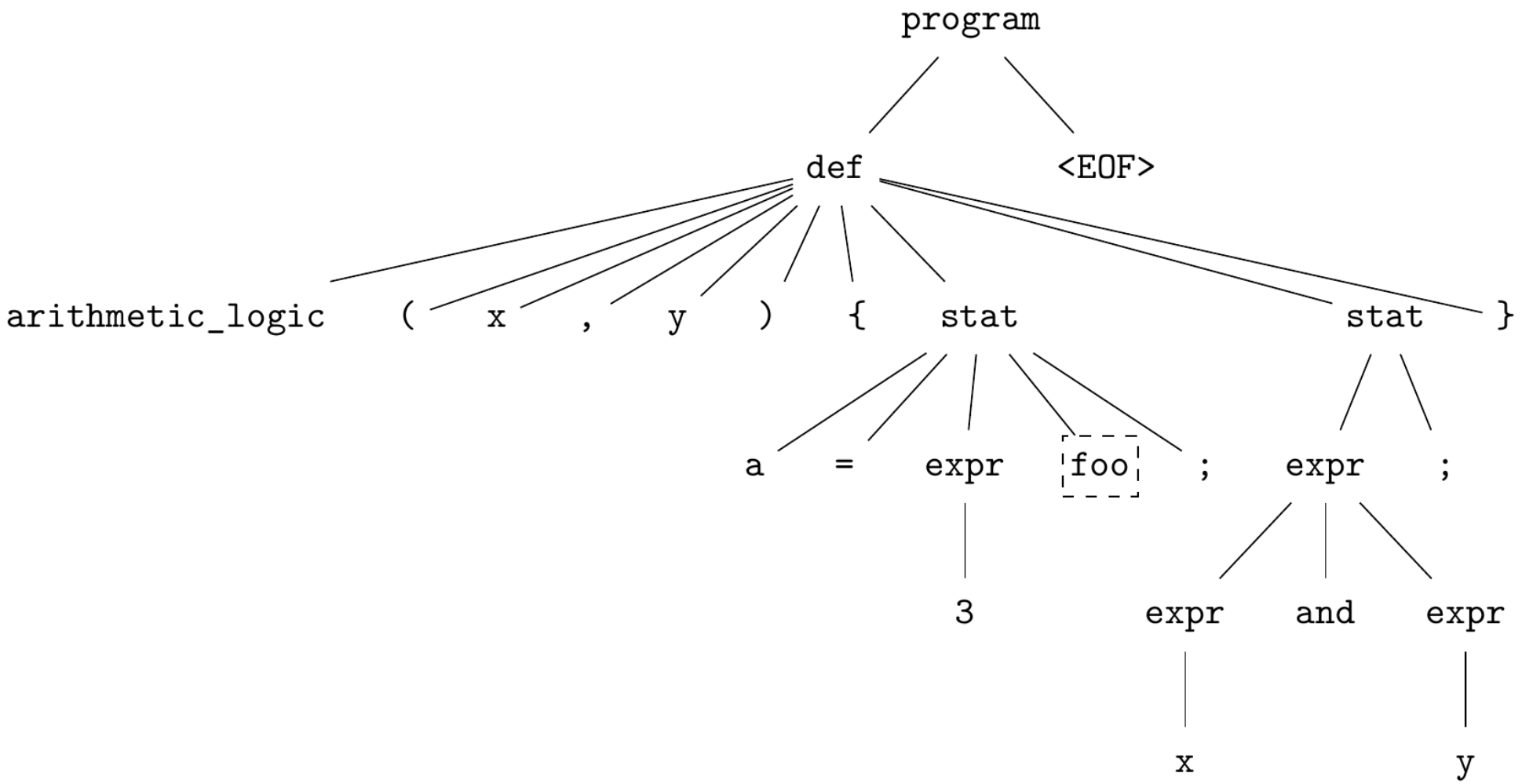
Lo que hace el parser de un lenguaje pequeño



Lo que hace el parser de un lenguaje pequeño



Lo que hace el parser de un lenguaje pequeño



Una gramática que produce ese árbol

```
stat ::= ID '=' expr ';'
       | expr ';'

expr ::= expr 'and' expr
       | INT
       | ID
```

Solo las reglas de stat y expr. Es una gramática posible, no la del curso.

¿Y si la gramática sí tuviera +?

```
fun arithmetic_logic(x,y) {
  a = 3+foo;
  x and y;
}
```

¿Y si la gramática sí tuviera +?

```
fun arithmetic_logic(x,y) {
  a = 3+foo;
  x and y;
}
```

1. foo tiene que estar ligado a algo (alcance)
2. 3+foo tiene que sumar dos números (tipos)
3. x y y tienen que ser booleanos (tipos)

¿Y si la gramática sí tuviera +?

```
fun arithmetic_logic(x,y) {
  a = 3+foo;
  x and y;
}
```

1. foo tiene que estar ligado a algo (alcance)
2. 3+foo tiene que sumar dos números (tipos)
3. x y y tienen que ser booleanos (tipos)
4. ¿a necesita declararse antes? Depende del lenguaje

La regla que falta

```
expr ::= expr 'and' expr
       | expr '+' expr
       | INT
       | ID
```

Sintaxis y semántica

Un texto es un programa de L si sigue la sintaxis de L.

Es un programa con sentido si además cumple la semántica de L.

Sin fijar L, la pregunta no tiene respuesta.

La misma línea en siete intérpretes

```
> a = b + c
```

a = b + c

Lenguaje	Respuesta	Cuándo se detecta
JavaScript		
Ruby		
Python		
Elixir		
Haskell		
C++		
Prolog		

a = b + c

Lenguaje	Respuesta	Cuándo se detecta
JavaScript	`ReferenceError: b is not defined`	al ejecutar
Ruby	`NameError: undefined local variable or method 'b'`	al ejecutar
Python	`NameError: name 'b' is not defined`	al ejecutar
Elixir		
Haskell		
C++		
Prolog		

a = b + c

Lenguaje	Respuesta	Cuándo se detecta
JavaScript	`ReferenceError: b is not defined`	al ejecutar
Ruby	`NameError: undefined local variable or method 'b'`	al ejecutar
Python	`NameError: name 'b' is not defined`	al ejecutar
Elixir	`undefined variable "b" (CompileError)`	antes de ejecutar
Haskell	`Variable not in scope: b (y también c)`	antes de ejecutar
C++		
Prolog		

a = b + c

Lenguaje	Respuesta	Cuándo se detecta
JavaScript	`ReferenceError: b is not defined`	al ejecutar
Ruby	`NameError: undefined local variable or method 'b'`	al ejecutar
Python	`NameError: name 'b' is not defined`	al ejecutar
Elixir	`undefined variable "b" (CompileError)`	antes de ejecutar
Haskell	`Variable not in scope: b (y también c)`	antes de ejecutar
C++	`Use of undeclared identifier 'a'`	antes de ejecutar
Prolog		

a = b + c

Lenguaje	Respuesta	Cuándo se detecta
JavaScript	`ReferenceError: b is not defined`	al ejecutar
Ruby	`NameError: undefined local variable or method 'b'`	al ejecutar
Python	`NameError: name 'b' is not defined`	al ejecutar
Elixir	`undefined variable "b" (CompileError)`	antes de ejecutar
Haskell	`Variable not in scope: b (y también c)`	antes de ejecutar
C++	`Use of undeclared identifier 'a'`	antes de ejecutar
Prolog	`false.`	al ejecutar

Prolog lee = como unificación

```
?- a = b + c.
false.
```

- a es un átomo
- b + c es el término compuesto +(b, c)
- Un átomo solo unifica consigo mismo o con una variable

Un error de sintaxis de verdad

```
print("hola")
a = = b
```

```
print("hola")
a = b + c
```

Un error de sintaxis de verdad

```
print("hola")
a = = b
```

```
  File "t.py", line 2
    a = = b
        ^
SyntaxError: invalid syntax
```

```
print("hola")
a = b + c
```

```
hola
Traceback (most recent call last):
  File "t2.py", line 2
    a = b + c
        ^
NameError: name 'b' is not defined
```

¿Es mejor detectar antes o al ejecutar?

Parte 2

Gramáticas

Una gramática es una 4-tupla

$$G = \langle N, \Sigma, S_0, P \rangle$$

Componente	**Qué es**
N	No terminales: las categorías sintácticas (variables)
Σ	Terminales: el alfabeto de símbolos con los que se escribe
S_0	Símbolo inicial, $S_0 \in N$
P	Reglas de producción: $(N \cup \Sigma)^* \rightarrow (N \cup \Sigma)^*$

Alfabetos y palabras

```
Σ_spanish = {a, b, c, …, z}
Σ_numbers = {0, 1, 2, …, 9}
```

- Una palabra ω de longitud m es una concatenación de m símbolos de Σ: $\omega \in \Sigma^m$
- λ (o ε) es la palabra vacía, de longitud 0
- Σ* es el conjunto de todas las palabras sobre Σ

Alfabetos y palabras

```
Σ_spanish = {a, b, c, …, z}
Σ_numbers = {0, 1, 2, …, 9}
```

¿"iokdafpodsaf" es una palabra?

Un lenguaje es un subconjunto de Σ*

$L \subseteq \Sigma^*$

- Los binarios pares: palabras sobre {0, 1} que terminan en 0
- {λ}: el lenguaje con una sola palabra, la vacía
- ø: el lenguaje sin palabras

¿{λ} y ø son el mismo lenguaje?

BNF: Backus-Naur Form

```
S ::= "a" S "b" | "ab"
```

- Los terminales van entre comillas; los no terminales, sin comillas
- ::= separa el lado izquierdo de la regla del derecho
- | separa alternativas del mismo no terminal

BNF: Backus-Naur Form

```
S ::= "a" S "b" | "ab"
```

S ⇒ ab

S ⇒ aSb ⇒ aabb

S ⇒ aSb ⇒ aaSbb ⇒ aaabbb

L = { $a^n b^n$ | n ≥ 1 }

EBNF: BNF extendido

Notación	Significado
α | β	α o β
α*	cero o más repeticiones de α
α+	una o más repeticiones de α
α?	α es opcional
(α β)	agrupar

La gramática de las listas

```
LIST   ::= "[" "]"
         | NUMBER "|" LIST

NUMBER ::= ("0" | "1" | … | "9")+
```

- [] es una lista
- Un número, seguido de |, seguido de una lista, es una lista

¿Cuáles son listas?

Texto	¿Lista?	Por qué
`3 | 14 | []`		
`[]`		
`[-7, 3, 14]`		
`[1 | "hello" | 3.14 | True]`		
`1 | [] | 7 | 9 | 3 | []`		
`1 | [] | [7 | 9 | 3 ]`		

¿Cuáles son listas?

Texto	¿Lista?	Por qué
`3 | 14 | []`	Sí	NUMBER | NUMBER | []
`[]`	Sí	La lista vacía
`[-7, 3, 14]`	No	El - y las comas no son terminales
`[1 | "hello" | 3.14 | True]`	No	Las comillas, las letras y el punto no son terminales
`1 | [] | 7 | 9 | 3 | []`	No	Después de [] no puede venir nada
`1 | [] | [7 | 9 | 3 ]`	No	Lo mismo, y [7 | 9 | 3] no es una lista

Sintaxis concreta y sintaxis abstracta

```
expr    ::= literal | unary | binary
unary   ::= "¬" expr
binary  ::= expr "*" expr | expr "/" expr
          | expr "+" expr | expr "-" expr
literal ::= "0".."9" | "true" | "false"
```

```
data Expr
  = var(Id id)
  | integer(int intValue)
  | \true()
  | \false()
  | not(Expr arg)
  | mul(Expr lhs, Expr rhs)
  | div(Expr lhs, Expr rhs)
  | add(Expr lhs, Expr rhs)
  | sub(Expr lhs, Expr rhs)
```

¿Qué pasa con estas expresiones?

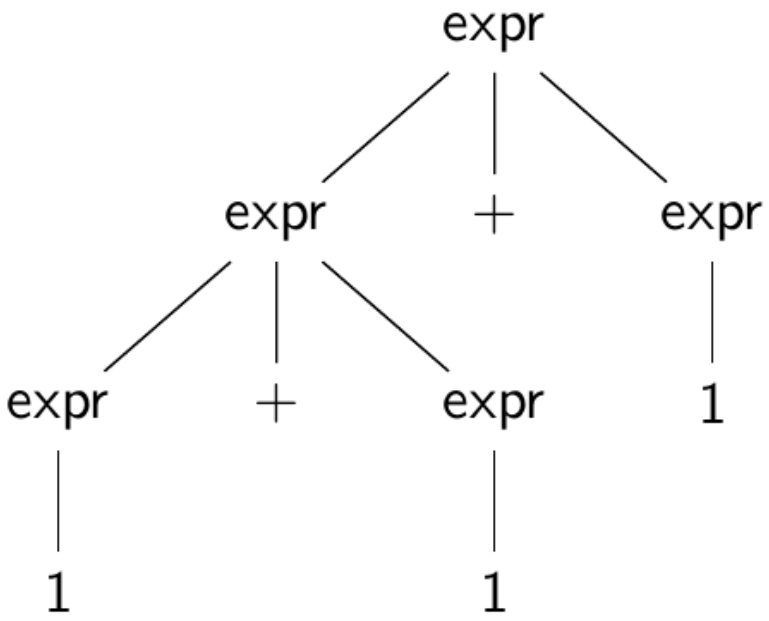
```
> 3 * 4
> ¬ false + true
> 1 + 1 + 1
```

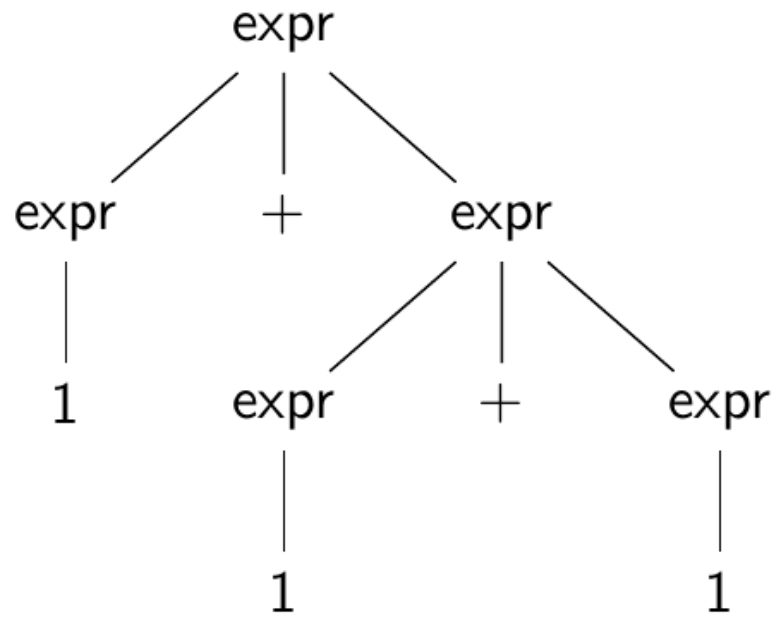
¿Qué pasa con estas expresiones?

```
> 3 * 4
> ¬ false + true
> 1 + 1 + 1
```

- 3 * 4: correcta
- ¬ false + true: sintaxis correcta, pero no es un programa bien formado
- 1 + 1 + 1: correcta, pero tiene dos árboles

1 + 1 + 1 tiene dos árboles





Un lenguaje pequeño: la hoja de cálculo

- Una hoja tiene celdas; una celda tiene un valor, una referencia o una fórmula
- Referencias estilo A1: columna y fila. $ hace absoluta la parte que sigue: $D7
- Rangos para funciones: SUM(A1:A5) suma A1, A2, A3, A4 y A5

```
=SUM(A1:A5)
=B7 + 3
=AVERAGE(AB$13, 9, 3.14, Z1480)
9
```

Alfabeto y categorías

```
Σ = {=, $, *, +, /, (, ), -, :, SUM, AVERAGE, MOD, ., 0..9, A..Z}
```

```
N = {ss, value, formulas, operators, functions,
     compound-formulas, simple-formulas,
     enumerated-formulas, range-formulas, references,
     relative-reference, fixed-reference, digit, letter}
```

La gramática de la clase

```
ss                ::= value | formula
value             ::= digit+
digit             ::= 0 | 1 | … | 9
letter            ::= A..Z
formula           ::= = internal-formula
internal-formula ::= value | ref | simple-formula
                    | compound-formula | enumerated-formula
ref               ::= $? letter letter? $? digit digit?
simple-formula    ::= internal-formula operation internal-formula
compound-formula ::= enumerated-formula | function(ref:ref)
enumerated-formula ::= function((ref|value) (,(ref|value))*)
function          ::= MOD | SUM | AVERAGE
```

Derivación de =B7 + 3

```
ss
⇒ formula
⇒ = internal-formula
⇒ = simple-formula
```

Derivación de =B7 + 3

```
ss
⇒ formula
⇒ = internal-formula
⇒ = simple-formula
⇒ = internal-formula operation internal-formula
⇒ = ref operation internal-formula
⇒ = B7 operation internal-formula
⇒ = B7 + internal-formula
⇒ = B7 + value
⇒ = B7 + 3
```

¿Cuáles genera la gramática?

Fórmula	¿Sí?	Por qué
`=SUM(A1:A5)`		
`=B7 + 3`		
`=AVERAGE(AB$13, 9, 3.14, Z1480)`		
`9`		
`=MOD(M9,L)`		
`A5`		
`=A3 ++ 7`		
`=AVERAGE(A1:A5)SUM(A1:A5)`		
`=SUM(A1,A2,)`		
`=3A + B5`		

¿Cuáles genera la gramática?

Fórmula	¿Sí?	Por qué
`=SUM(A1:A5)`	Sí	compound-formula: función sobre un rango
`=B7 + 3`	Sí	simple-formula
`=AVERAGE(AB$13, 9, 3.14, Z1480)`	No	3.14 no es value; Z1480 tiene 4 dígitos
`9`	Sí	value
`=MOD(M9,L)`	No	L no es ref: le falta el dígito
`A5`	No	Una ref sola no es ss: le falta el =
`=A3 ++ 7`	No	Dos operaciones seguidas
`=AVERAGE(A1:A5)SUM(A1:A5)`	No	Dos funciones sin operación en medio
`=SUM(A1,A2,)`	No	Coma al final
`=3A + B5`	No	3A no es value ni ref

Parte 3

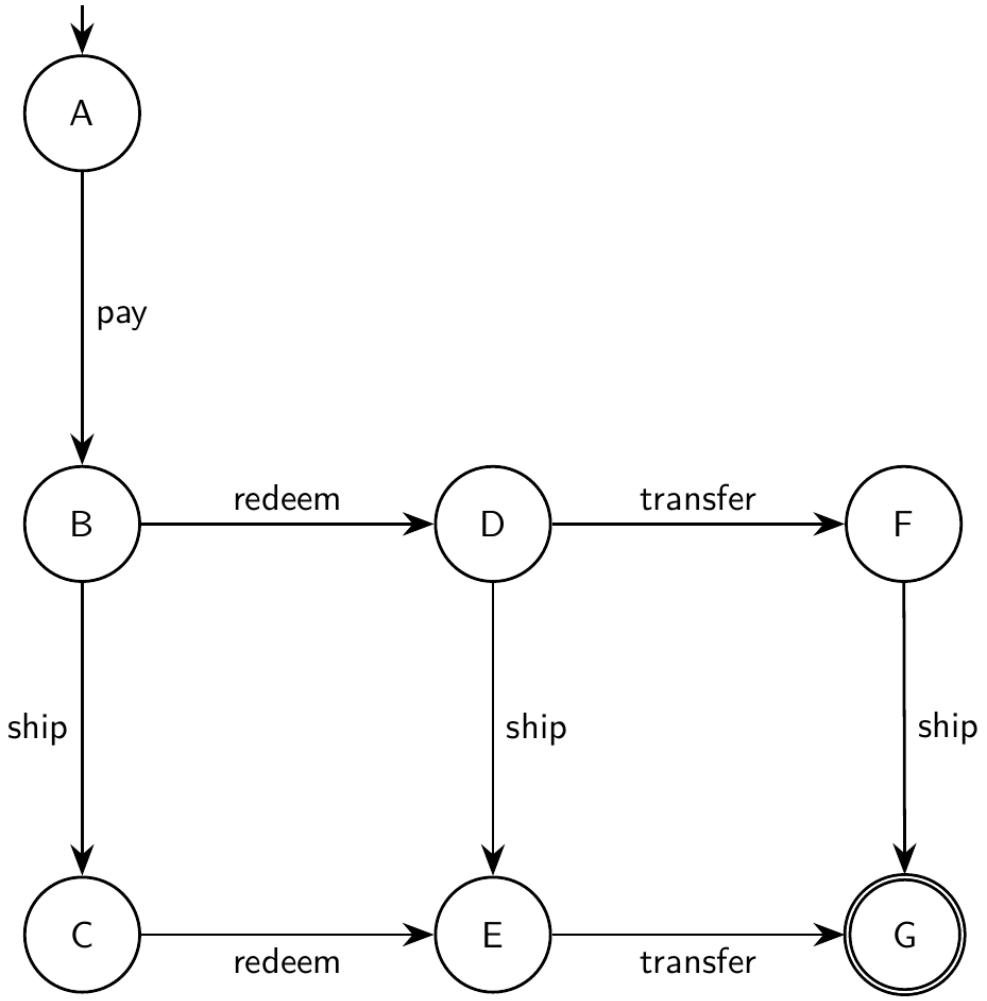
Autómatas: DFA y NFA

Sistemas de transiciones etiquetadas

$\mathcal{M} = \langle Q, \Sigma, \delta \rangle$

Componente	**Qué es**
Q	Los estados del sistema
Σ	Las acciones (etiquetas) posibles
$\delta : Q \times \Sigma \to Q$	Las transiciones: desde un estado, con una acción, a otro estado

Ejemplo: una compra



- pay: el cliente paga
- ship: la tienda envía
- redeem: la tienda cobra en el banco
- transfer: el banco transfiere

Autómata finito determinista (DFA)

$$\mathcal{A} = \langle Q, \Sigma, q_I, F, \delta \rangle$$

Componente	Qué es
Q	Conjunto finito de estados
Σ	Alfabeto de entrada
$q_I \in Q$	Estado inicial
$F \subseteq Q$	Estados finales
$\delta : Q \times \Sigma \to Q$	Función de transición

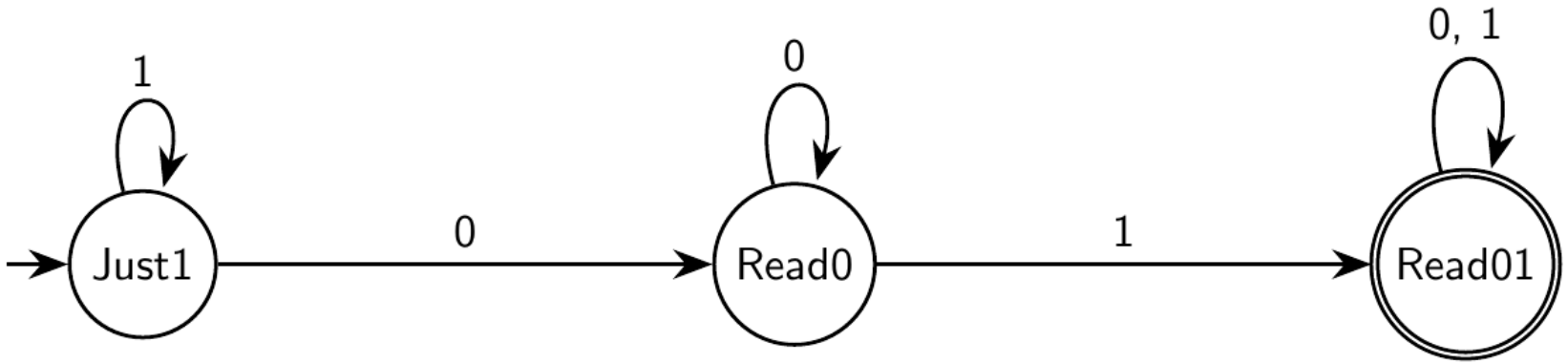
Cuándo un DFA acepta una palabra

$\omega = \sigma_1\sigma_2\ldots\sigma_n$ es aceptada si hay estados $q_0, \ldots, q_n$ con $q_0 = q_I$, $q_n \in F$ y $\delta(q_{i-1}, \sigma_i)$ $= q_i$ para cada i.

$\hat{\delta} : Q \times \Sigma^* \to Q$

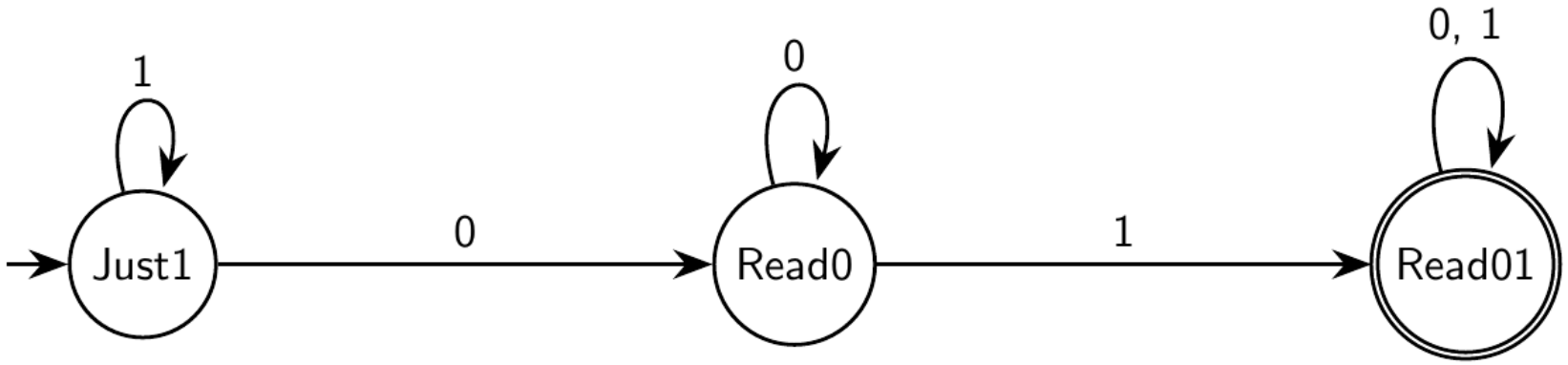
$L(\mathscr{A}) = \{ \omega \mid \hat{\delta}(q_I, \omega) \in F \}$

Palabras que contienen 01




```
01
1010
100011
0
111000
```

Palabras que contienen 01

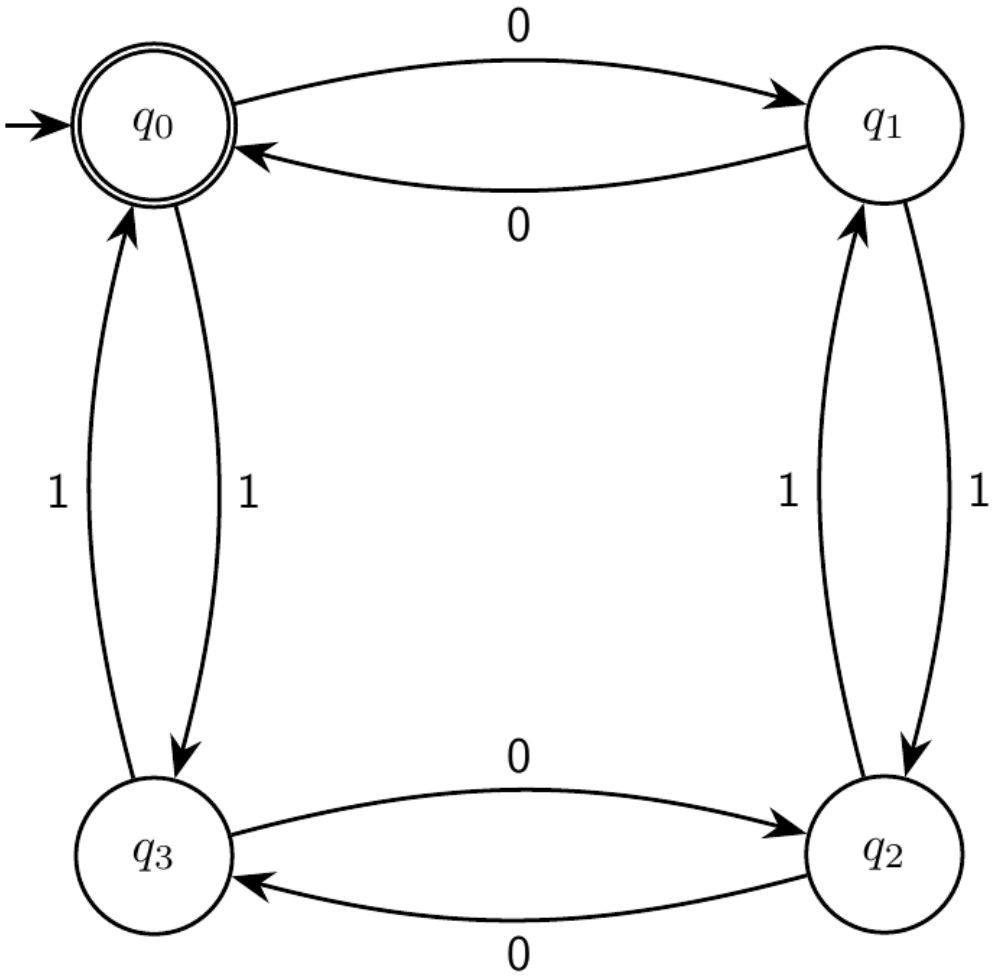



```
01      sí
1010    sí
100011  sí
0       no
111000  no
```

Ejercicio 1

Define un DFA que acepte las palabras sobre {0, 1} con un número par de 0 y un número par de 1.

Ejercicio 1: solución



impar 0
par 1

par 0
impar 1

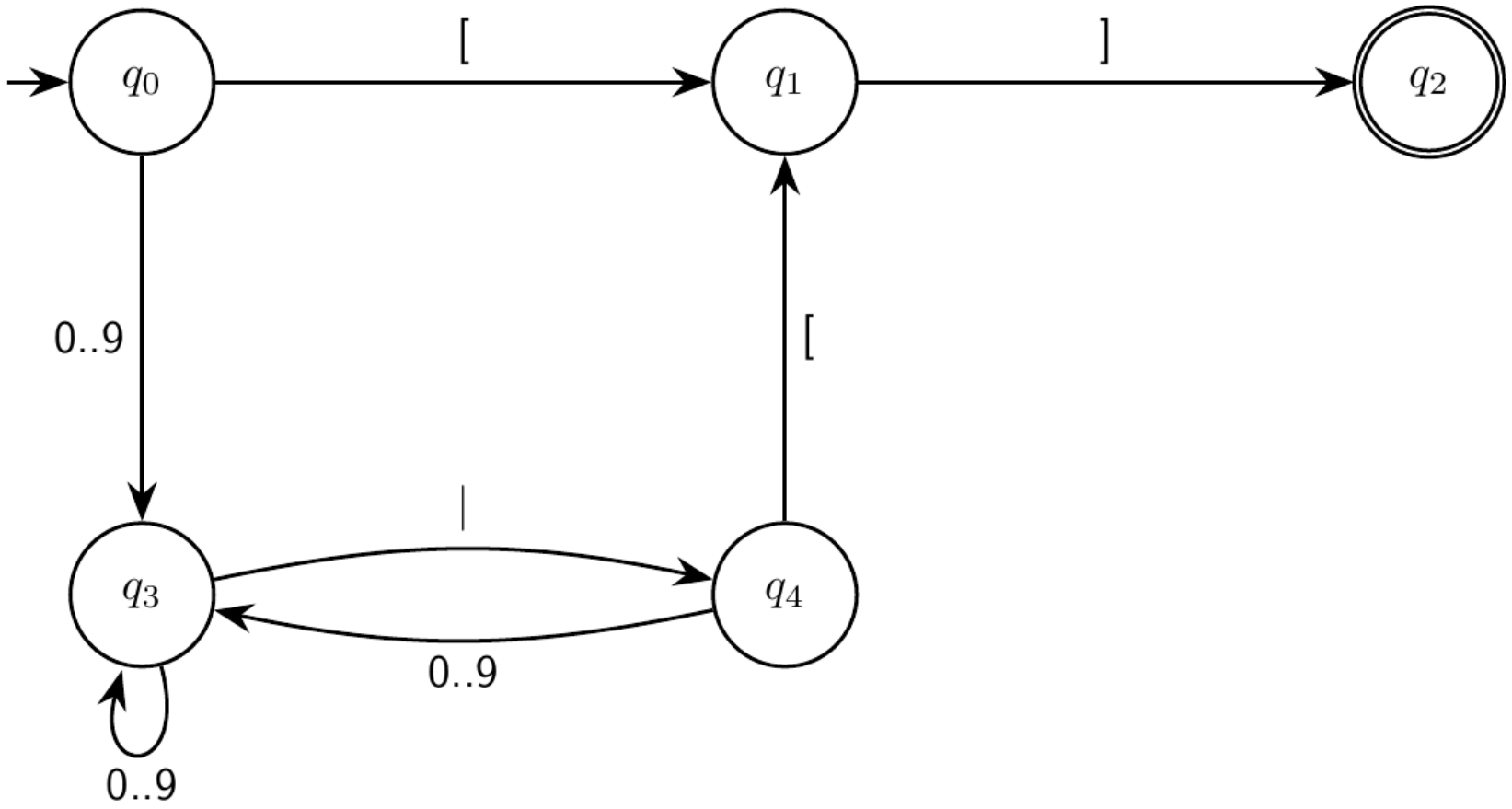
impar 0
impar 1

Ejercicio 2

Define un DFA que acepte las palabras del lenguaje de las listas.

```
[]
1|[]
1|2|3|[]
```

Ejercicio 2: solución



Ejercicio 2: el estado de error

- Se agrega un estado err, que no es final
- Toda transición que falta va a err
- err va a err con cualquier símbolo

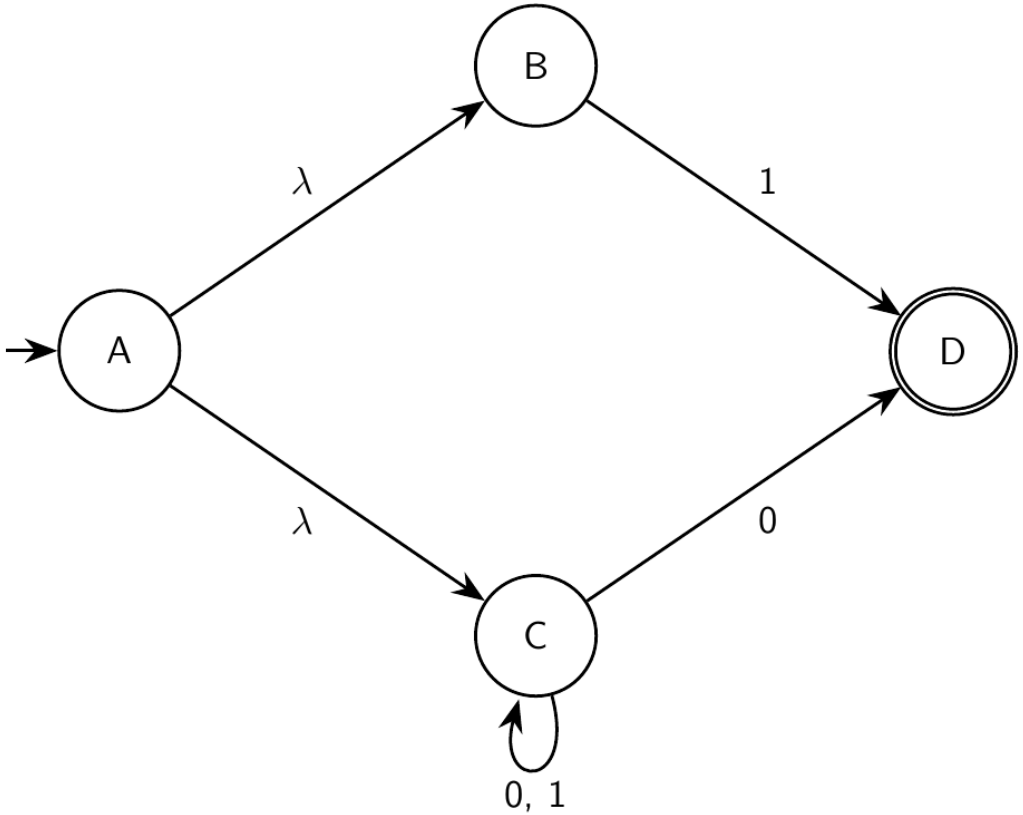
Autómata finito no determinista (NFA)

$\mathscr{A} = \langle Q, \Sigma, I, F, \Delta \rangle$

Componente	Qué cambia frente al DFA
$I \subseteq Q$	Un conjunto de estados iniciales, no uno solo
$\Delta \subseteq Q \times (\Sigma \cup \{\lambda\}) \times Q$	Una relación, no una función

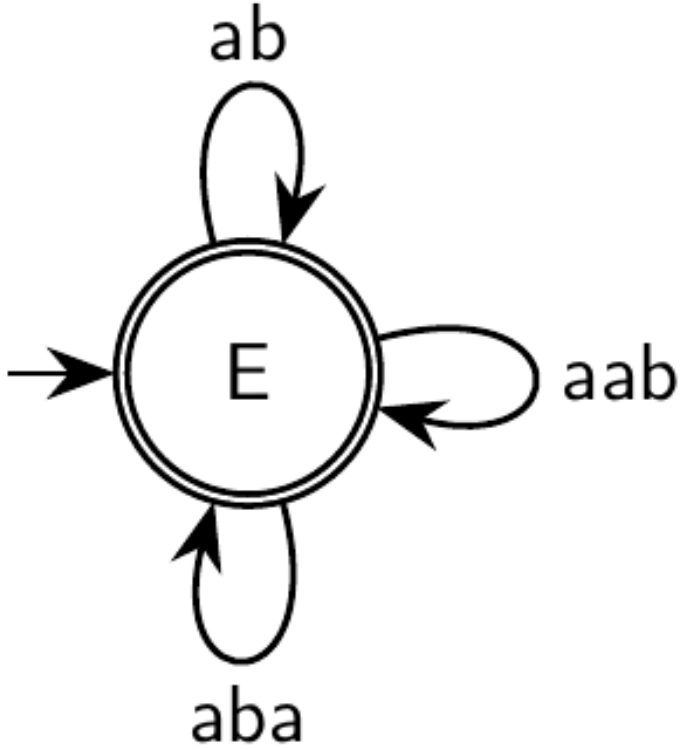
- Transiciones con λ (sin leer nada) o sobre palabras
- Varias transiciones con el mismo símbolo
- Varios estados iniciales

Cuándo un NFA acepta una palabra



Acepta ω si existe algún camino de un estado inicial a uno final que lea toda ω.

Ejercicio: (ab | aab | aba)*



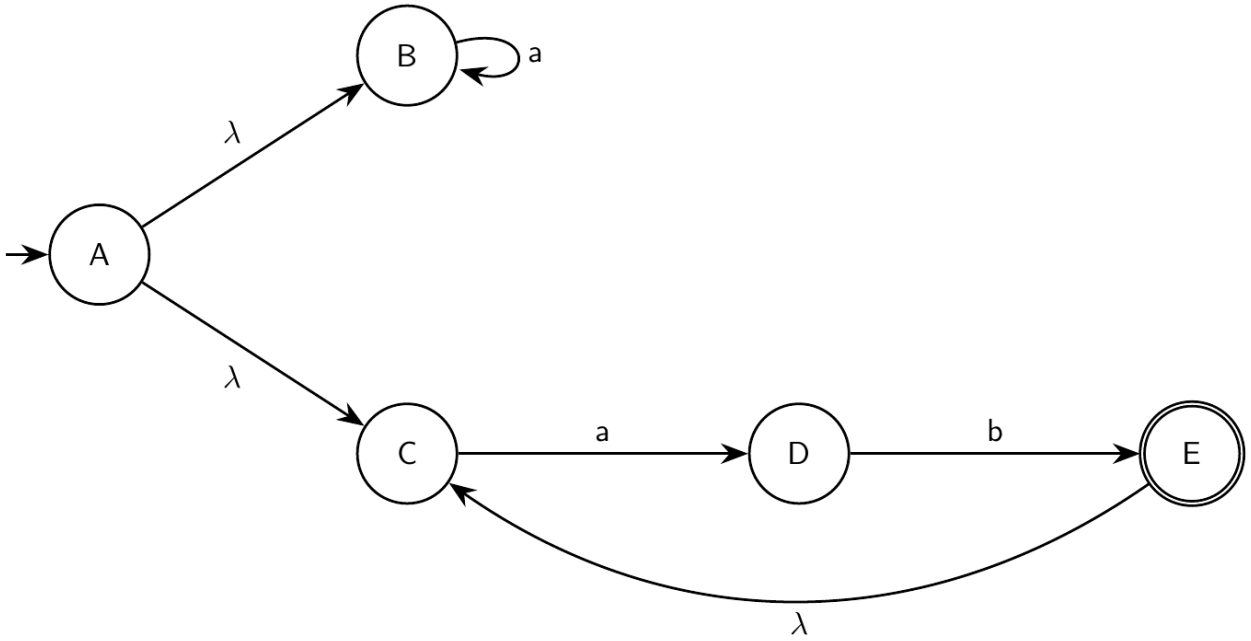
¿Los NFA son más poderosos que los DFA?

Teorema: todo NFA tiene un DFA equivalente.

1. Eliminar las transiciones λ
2. Eliminar la elección (varias flechas con el mismo símbolo)
3. Dejar un solo estado inicial

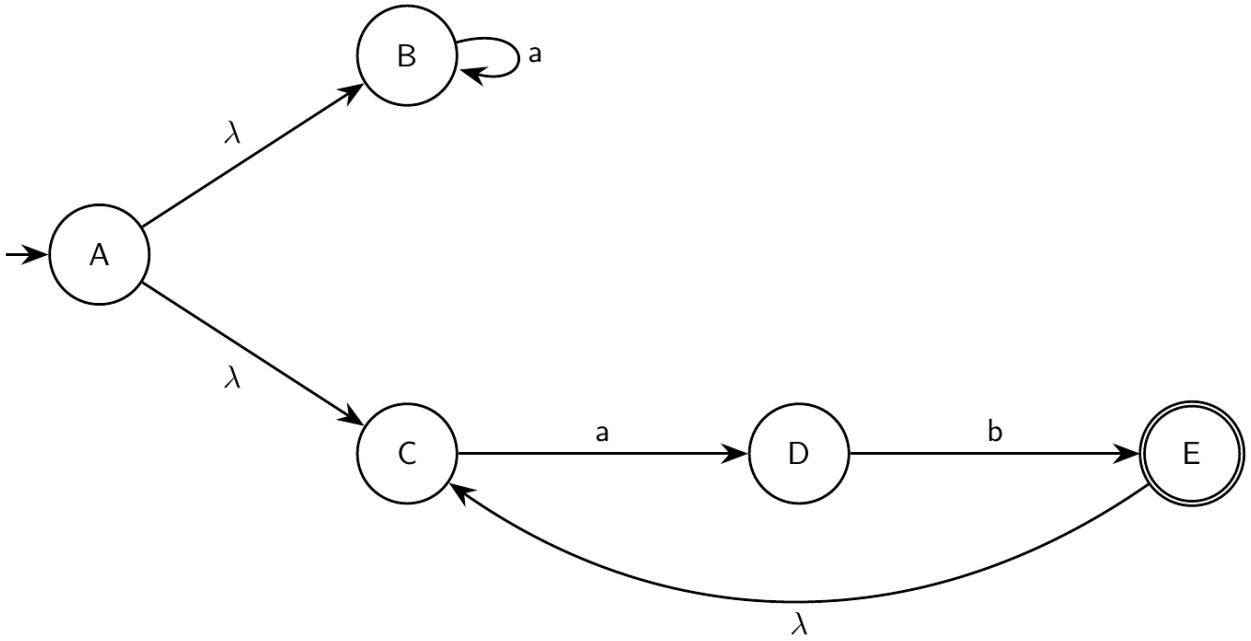
En los tres pasos, un estado del DFA es un conjunto de estados del NFA.

Clausura λ



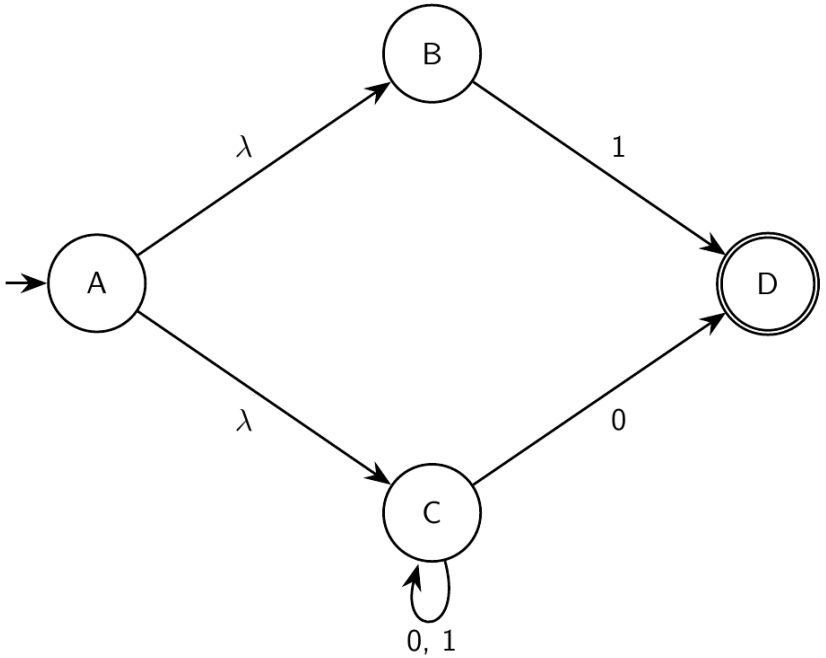
cl_λ(q): los estados a los que se llega desde q sin leer nada. ¿cl_λ(A)? ¿cl_λ(E)?

Clausura λ



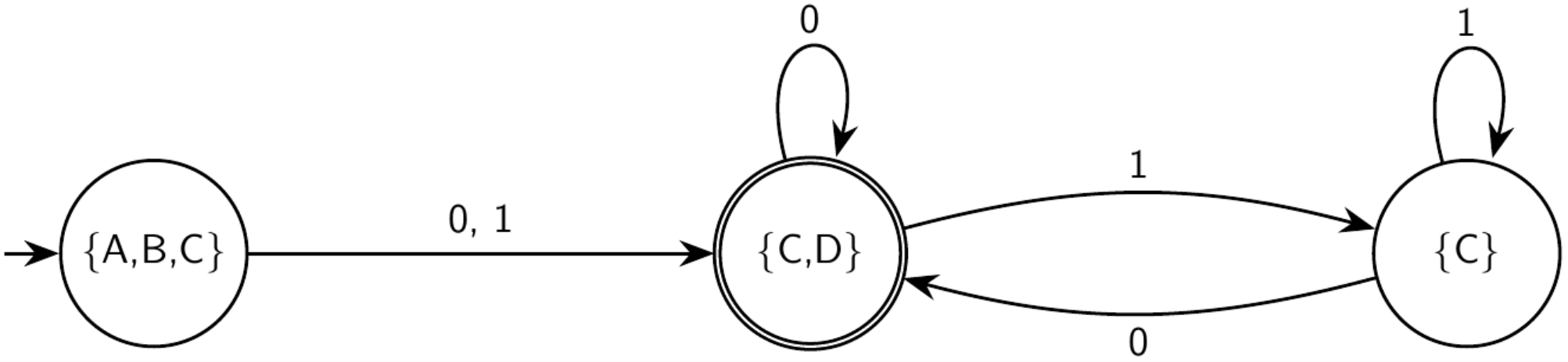
cl_λ(A) = {A, B, C}
cl_λ(E) = {E, C}

Construcción de subconjuntos

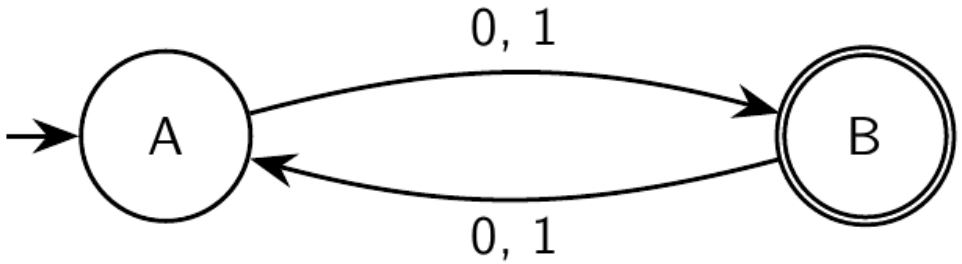


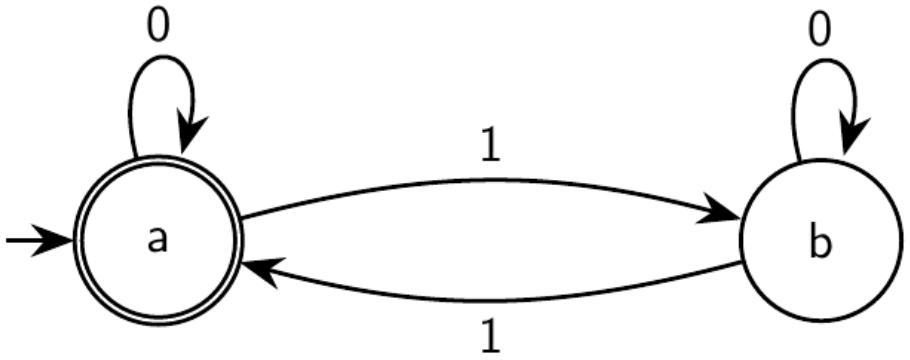
Estado del DFA	**0**	**1**
{A, B, C} (inicial)	{C, D}	{C, D}
{C, D} (final)	{C, D}	{C}
{C}	{C, D}	{C}

El DFA resultante



Combinar autómatas





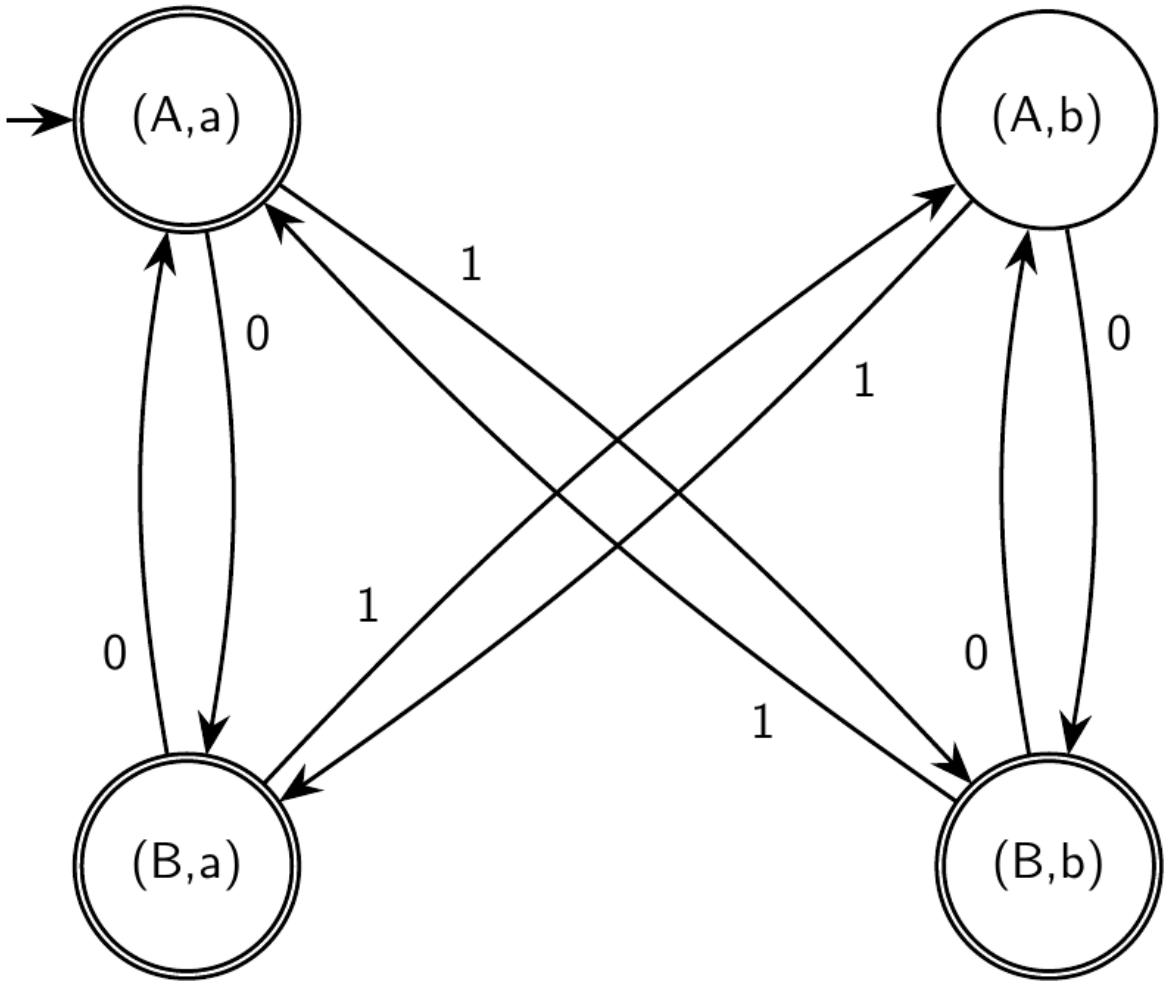
L_1: longitud impar

L_2: número par de 1

```
01011100
1101001
11110100
```

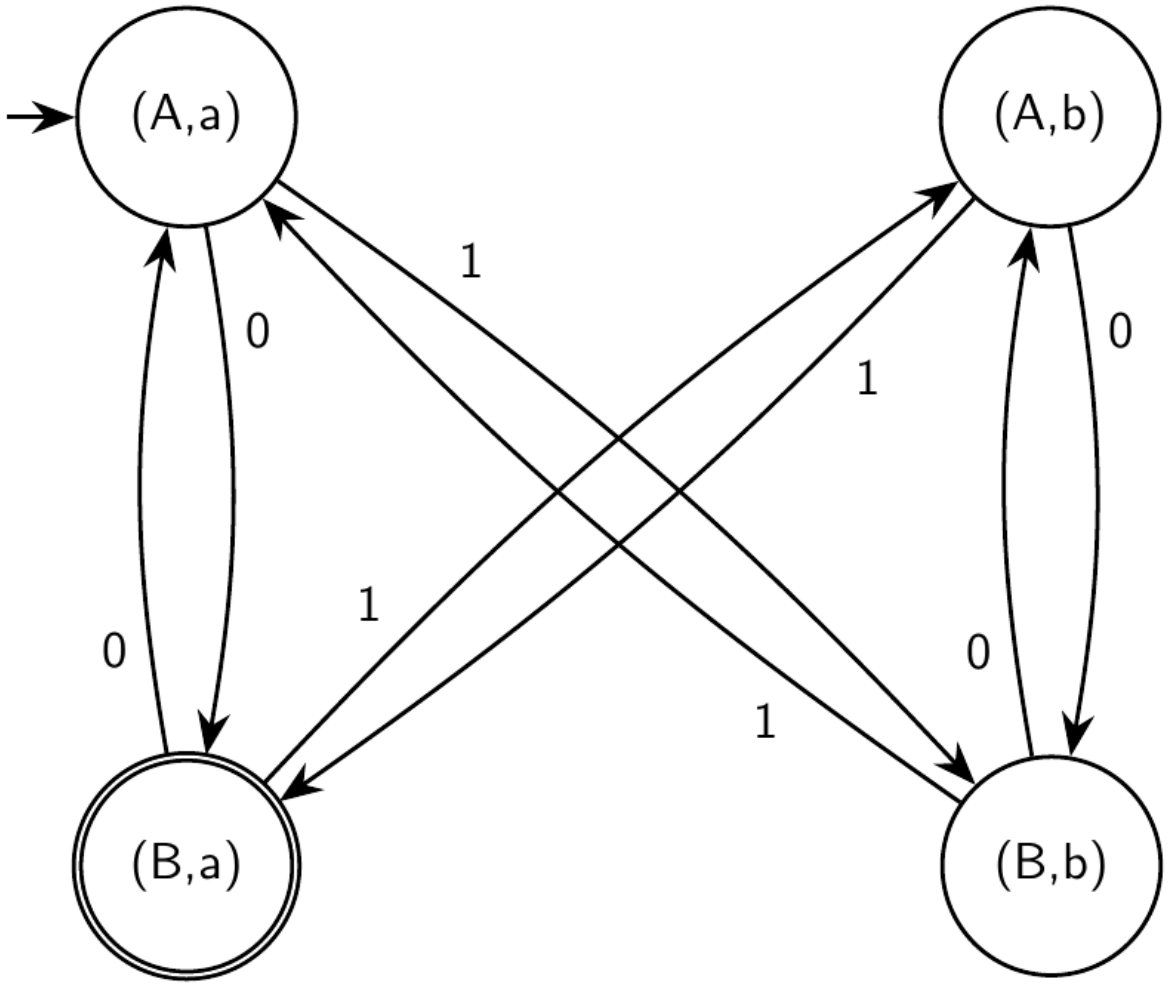
¿Cuáles están en L_1? ¿En L_2? ¿En los dos?

Unión: $L_1 \cup L_2$



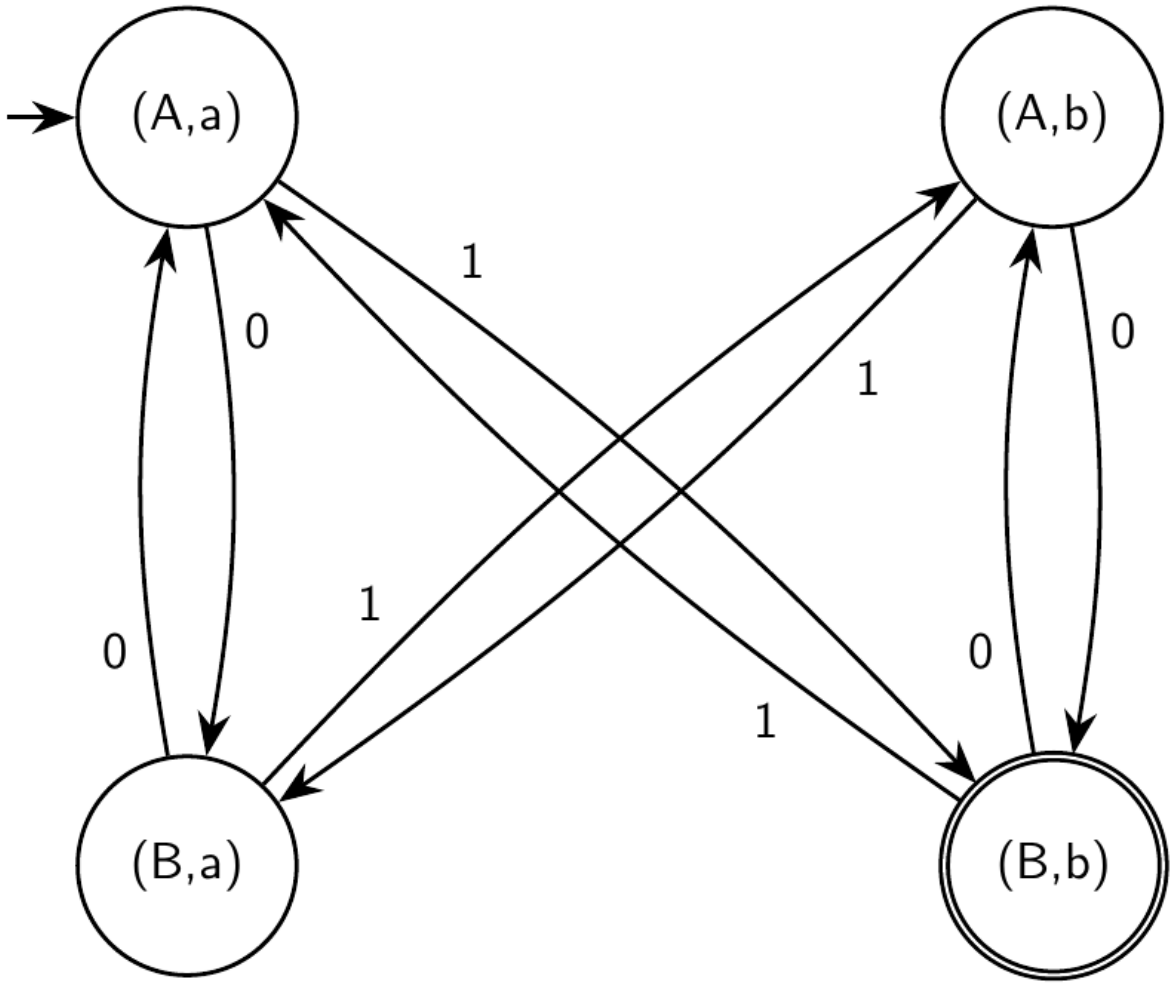
$$F = \{(p,q) \mid p \in F_1 \vee q \in F_2\}$$

Intersección: $L_1 \cap L_2$



$$F = \{(p,q) \mid p \in F_1 \land q \in F_2\}$$

Diferencia: $L_1 \setminus L_2$



$$F = \{(p,q) \mid p \in F_1 \wedge q \notin F_2\}$$

Las tres palabras

Palabra	**L_1**	**L_2**	**$\cup$**	$\cap$	****
`01011100`	no	sí	sí	no	no
`1101001`	sí	sí	sí	sí	no
`11110100`	no	no	no	no	no

- DFA y NFA reconocen los mismos lenguajes
- Los NFA son más fáciles de escribir
- Pero pueden aceptar palabras que uno no quería: hay que probarlos

Parte 4

Lenguajes regulares

Gramáticas regulares (tipo 3)

```
A ::= "a"
A ::= B "a"
```

Gramática izquierda

```
A ::= "a"
A ::= "a" B
```

Gramática derecha

A, B ∈ N, a ∈ Σ. Es un subconjunto de las gramáticas libres de contexto: tipo 3 ⊆ tipo 2.

Expresiones regulares

Expresión	**Significado**
λ	La palabra vacía
σ	Un símbolo $\sigma \in \Sigma$
α | β	α o β
αβ	α seguida de β
α*	Cero o más veces α (estrella de Kleene)
α+	Una o más veces α
α?	α opcional

Ojo con el punto

```
>>> "one.two.".replace(".", "*")
'one*two*'
>>> re.sub(".", "*", "one.two.")
'*********'
>>> re.sub(r"\.", "*", "one.two.")
'one*two*'
```

¿Cuál está mal?

Lenguaje	**Expresión de la clase**	
Contiene 00	`(0|1)*00(0|1)*`	
Dos a nunca seguidas, sobre {a, b}	`a(b|ba)* | (b|ba)*`	
Empieza con 0 y es impar, o empieza con 1 y es par	`0((0|1)(0|1))* | 1(0|1)*`	

¿Cuál está mal?

Lenguaje	**Expresión de la clase**	**Veredicto**
Contiene 00	`(0|1)*00(0|1)*`	Correcta
Dos a nunca seguidas, sobre {a, b}	`a(b|ba)* | (b|ba)*`	Correcta
Empieza con 0 y es impar, o empieza con 1 y es par	`0((0|1)(0|1))* | 1(0|1)*`	Acepta 1, 100, 101…

`Corrección: 0((0|1)(0|1))* | 1(0|1)((0|1)(0|1))*`

Ejercicio: correo de Uniandes

```
[A-Za-z0-9._-]+@uniandes\.edu\.co
```

Texto	¿Coincide?
`cf.gonzalez@uniandes.edu.co`	sí
`j-perez10@uniandes.edu.co`	sí
`cf.gonzalez@uniandes.edu.com`	no
`@uniandes.edu.co`	no

De expresiones a gramáticas

Expresión	Gramática derecha	Gramática izquierda
a	S ::= a	S ::= a
α | β	S ::= α | β	S ::= α | β
αβ	S ::= αA, A ::= β	S ::= Aβ, A ::= α
α*	S ::= αS | λ	S ::= Sα | λ
α+	S ::= αS | α	S ::= Sα | α
(α | β)*	S ::= αS | βS | λ	S ::= Sα | Sβ | λ

Gramática derecha: derivación de bbbabba

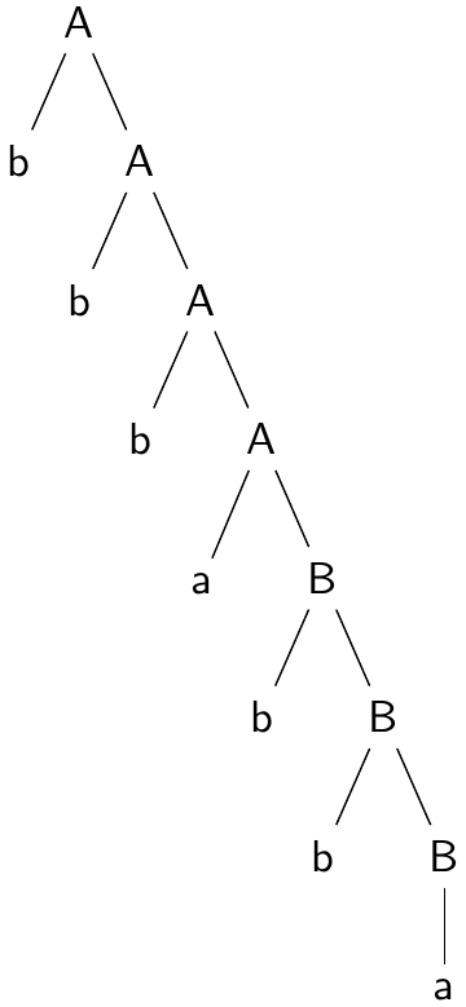
```
A ::= λ | aB | bA | b
B ::= bB | a
```

A ⇒ bA
⇒ bbA
⇒ bbbA

Gramática derecha: derivación de bbbabba

```
A ::= λ | aB | bA | b
B ::= bB | a
```

A ⇒ bA
⇒ bbA
⇒ bbbA
⇒ bbbaB
⇒ bbbabB
⇒ bbbabbB
⇒ bbbabba



Gramática izquierda

```
S ::= Abbb | Sb
A ::= Aa | aa
```

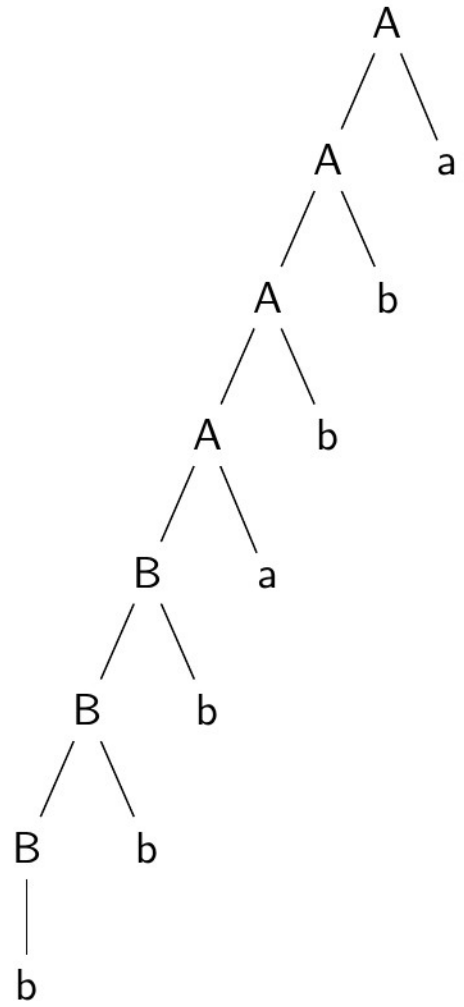
```
S ⇒ Abbb
  ⇒ aabbb
```

¿Cuál es la palabra más corta? ¿Cuál es el lenguaje?

El árbol de una gramática izquierda

```
A ::= a | Ab | Ba | b
B ::= a | b | Bb
```

```
A ⇒ Aa ⇒ Aba ⇒ Abba
  ⇒ Babba ⇒ Bbabba
  ⇒ Bbbabba ⇒ bbbabba
```



Quiz de la clase

```
s ::= (bc)*t
b ::= m+ | pbt
c ::= b | e+
```

1. ¿Cuáles son los no terminales?
2. ¿Cuáles son los terminales?
3. ¿Qué reglas son recursivas?

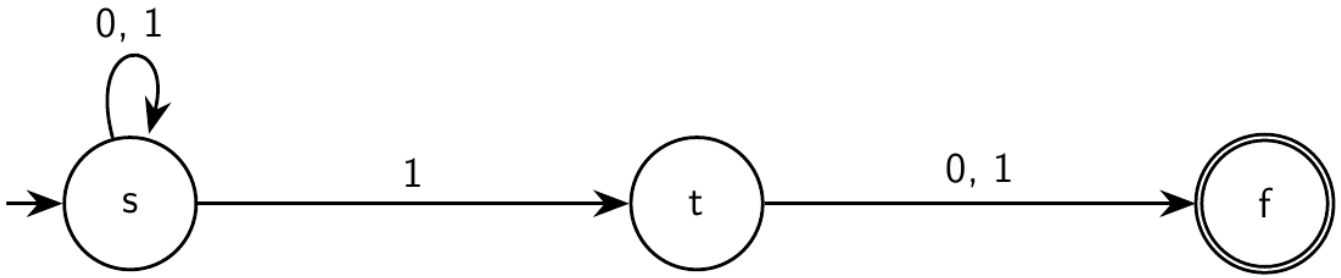
Quiz de la clase

```
s ::= (bc)*t
b ::= m+ | pbt
c ::= b | e+
```

Pregunta	**Respuesta**
No terminales	s, b, c: tienen regla
Terminales	e, m, p, t: aparecen y no tienen regla
Recursivas	b, por pbt
|, *, +	No son ni lo uno ni lo otro: son la notación de EBNF

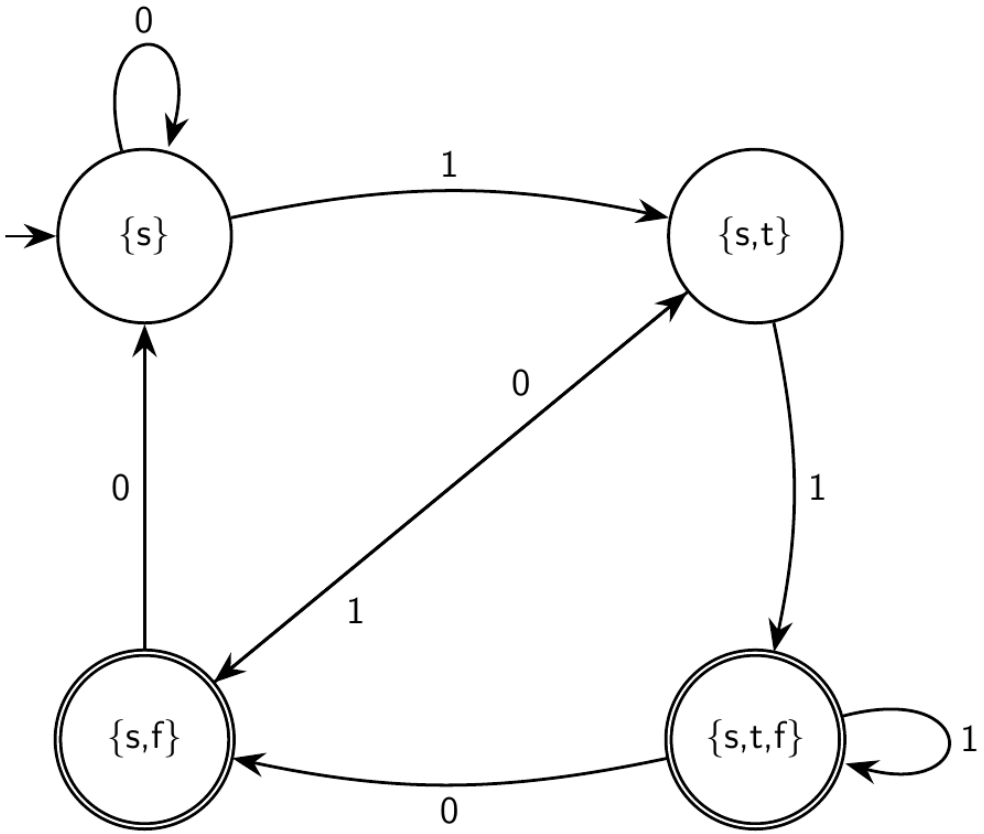
De expresión regular a DFA

`(0|1)*1(0|1)`



De expresión regular a DFA

`(0|1)*1(0|1)`



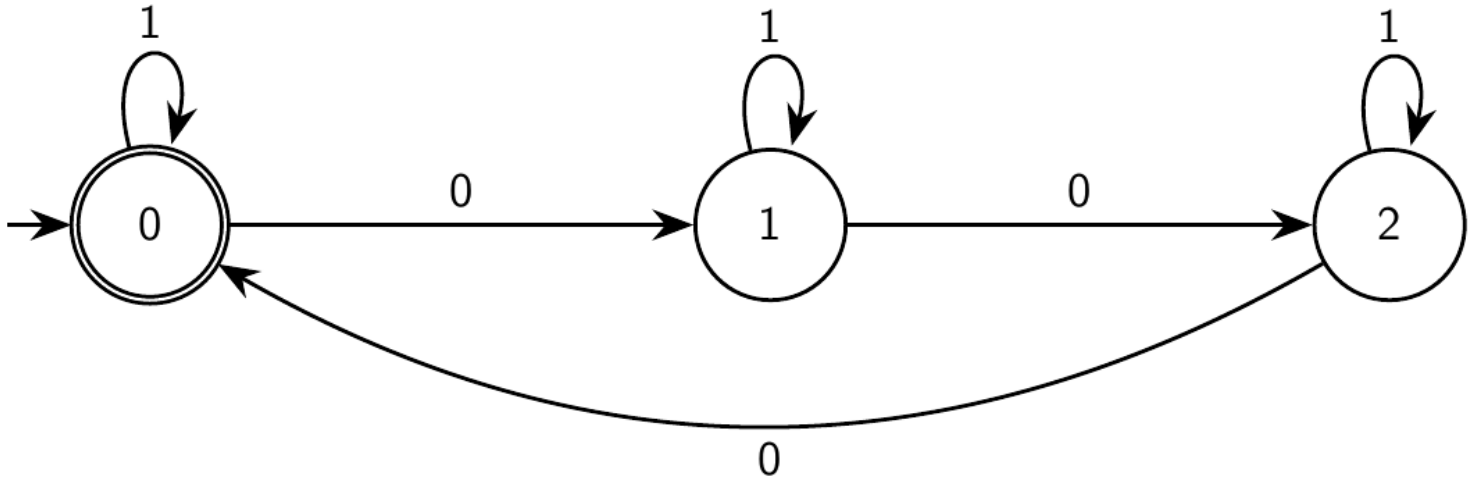
Ejercicio 5 de la clase

	Lenguaje sobre {0, 1}	**Expresión de la clase**	
1	Número de 0 divisible por 3	`(1 | 01*01*0)*`	
2	Número par de 0	`(1 | 01*0)*`	
3	El 5.º desde la derecha es 1	`(0|1)*1(0|1)(0|1)(0|1)(0|1)`	
4	Cada 0 seguido de al menos un 1	`(0?1)*`	
5	0^n1^m con $n \geq 3$ y m par	`000*0(11)*`	
6	A lo sumo dos 0	`1*0?1*0?1*`	
7	Empieza con 0 e impar, o con 1 y par	`(0|1(0|1))((0|1)(0|1))*`	
8	Todas menos 0 y 1	`(0|1)(0|1)(0|1)*`	
9	No termina en 01	`0 | 1 | (0|1)*(0|1(0|1))`	

Ejercicio 5 de la clase

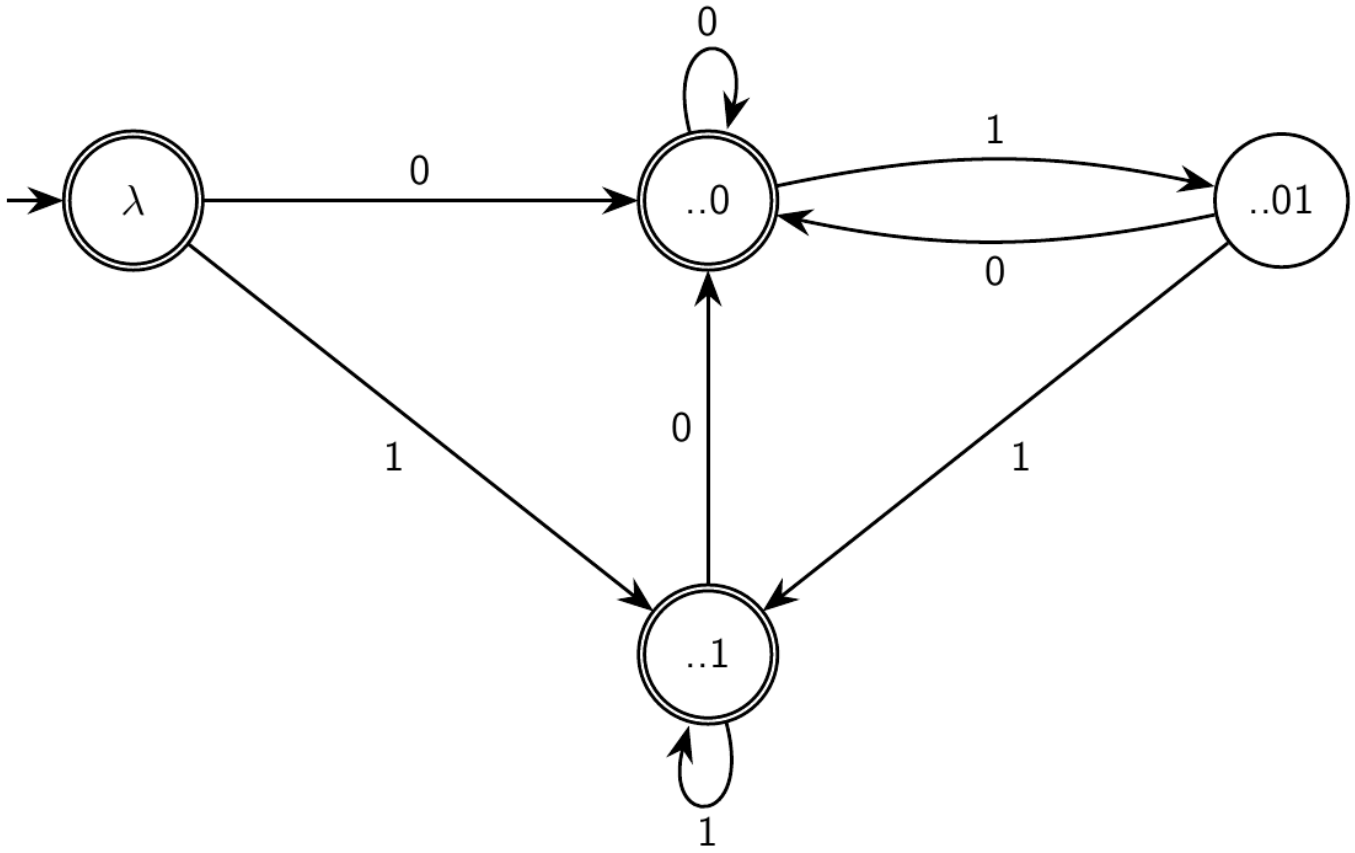
	Lenguaje sobre {0, 1}	**Expresión de la clase**	**Verificada**
1	Número de 0 divisible por 3	`(1 | 01*01*0)*`	Correcta
2	Número par de 0	`(1 | 01*0)*`	Correcta
3	El 5.º desde la derecha es 1	`(0|1)*1(0|1)(0|1)(0|1)(0|1)`	Correcta
4	Cada 0 seguido de al menos un 1	`(0?1)*`	Correcta
5	0^n1^m con n ≥ 3 y m par	`000*0(11)*`	Correcta
6	A lo sumo dos 0	`1*0?1*0?1*`	Correcta
7	Empieza con 0 e impar, o con 1 y par	`(0|1(0|1))((0|1)(0|1))*`	Correcta
8	Todas menos 0 y 1	`(0|1)(0|1)(0|1)*`	No acepta λ
9	No termina en 01	`0 | 1 | (0|1)*(0|1(0|1))`	No acepta λ

5.1: ceros divisibles por 3




```
δ(q, 0) = (q + 1) % 3
δ(q, 1) = q
```

5.9: no termina en 01



Cada estado recuerda lo último que se leyó.

Un lenguaje regular real: fechas

MM/DD/YYYY
DD-MM-YY
YYYY.MM.DD

```
Date   ::= Slash | Dash | Dot
Slash  ::= Month|Month1 / Day|Day1 / Year1 Year Year Year
Dash   ::= Day|Day1 - Month|Month1 - Year Year
Dot    ::= Year1 Year Year Year . Month|Month1 . Day|Day1
Day    ::= (0..2)(0..9)      Day1   ::= 3(0|1)
Month  ::= 0(1..9)           Month1 ::= 1(0..2)
Year1  ::= (0..2)            Year   ::= (0..9)
```

Derivación de 03/08/1978

```
Date ⇒ Slash
     ⇒ Month|Month1 / Day|Day1 / Year1 Year Year Year
     ⇒ 03 / Day|Day1 / Year1 Year Year Year
     ⇒ 03 / 08 / Year1 Year Year Year
     ⇒ 03 / 08 / 1 Year Year Year
     ⇒ 03 / 08 / 19 Year Year
     ⇒ 03 / 08 / 197 Year
     ⇒ 03 / 08 / 1978
```

¿Cuáles reconoce?

Texto	¿Sí?	Por qué
03/08/1978		
1979.10.01		
12-11-2025		
12-11-25		
02/31/2000		
13/01/2000		

¿Cuáles reconoce?

Texto	¿Sí?	Por qué
`03/08/1978`	sí	Slash
`1979.10.01`	sí	Dot
`12-11-2025`	no	Dash solo tiene años de dos dígitos
`12-11-25`	sí	Dash
`02/31/2000`	sí	¡31 de febrero!
`13/01/2000`	no	No hay mes 13

Lenguajes que no son regulares

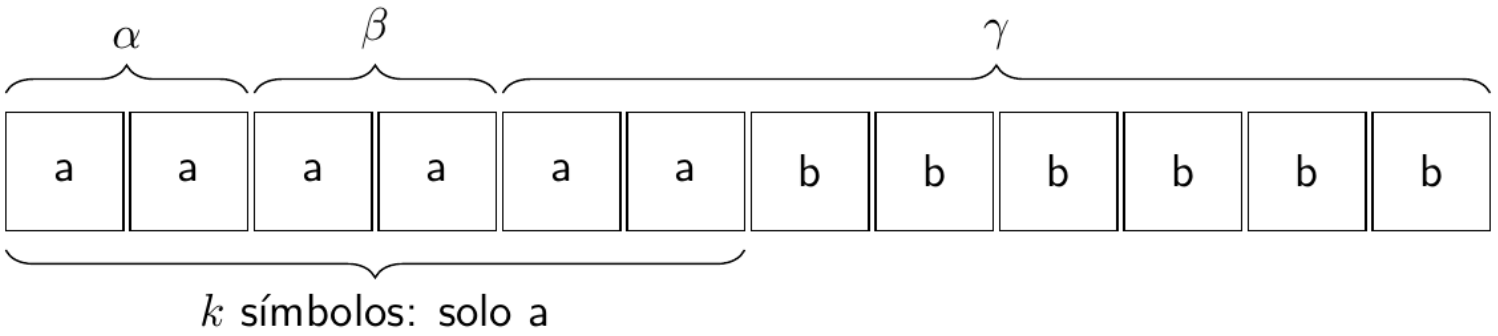
Lema de bombeo. Si L es regular e infinito, existe k tal que toda $\omega \in L$ con $|\omega| \geq k$ se puede escribir $\omega = \alpha\beta\gamma$ con $|\alpha\beta| \leq k$, $\beta \neq \lambda$, y $\alpha\beta^n\gamma \in L$ para todo $n \geq 0$.

Es una condición necesaria: si un lenguaje no la cumple, no es regular.

El lema como un juego

1. Te dan un lenguaje L y quieres probar que no es regular
2. El adversario escoge k
3. Tú escoges $\omega \in L$ con $|\omega| \geq k$
4. El adversario divide $\omega = \alpha\beta\gamma$ con $\beta \neq \lambda$ y $|\alpha\beta| \leq k$
5. Tú escoges n tal que $\alpha\beta^n\gamma \notin L$

$a^n b^n$ no es regular



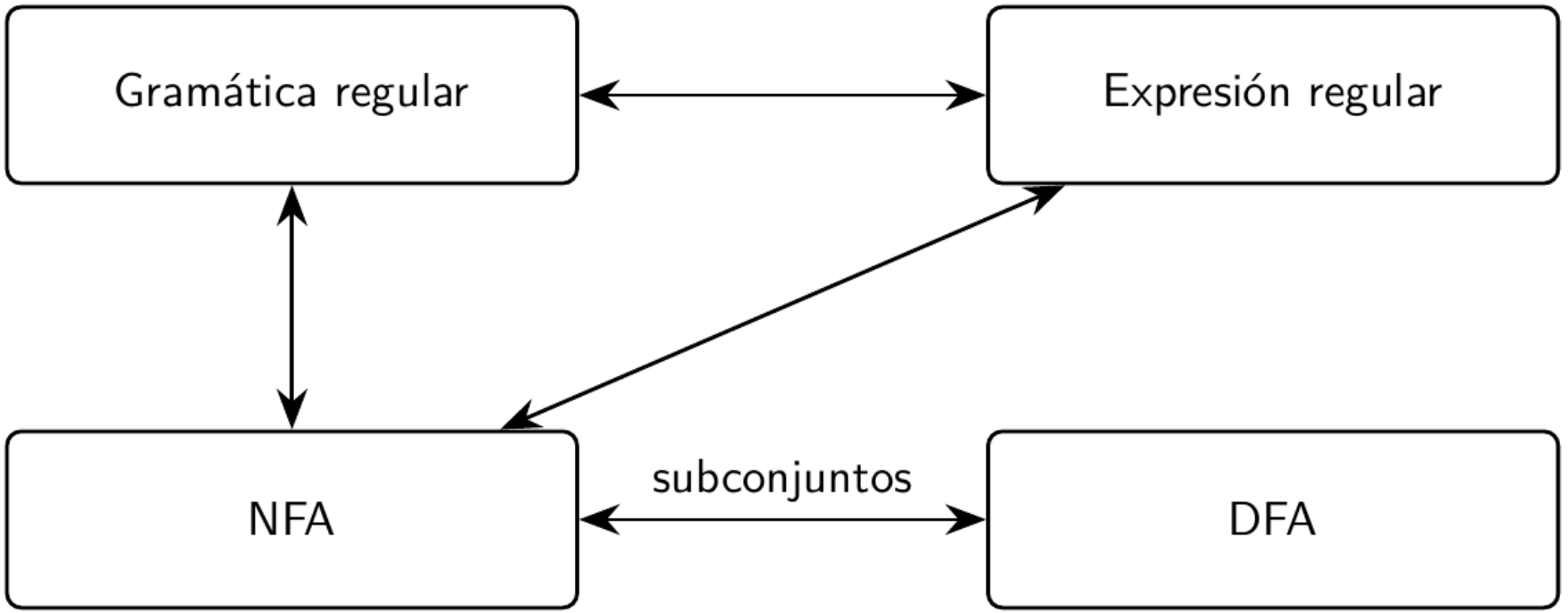
1. Escojo $\omega = a^k b^k$, que está en L y tiene longitud $2k \geq k$
2. Como $|\alpha\beta| \leq k$, α y β están dentro de las aes: $\beta = a^t$ con $t > 0$
3. Con n = 0 queda $a^{k-t} b^k$: menos aes que bes
4. $a^{k-t} b^k \notin L$. Contradicción: L no es regular

Propiedades de clausura

Si L_1 y L_2 son regulares, también lo son:

1. $L_1 \cup L_2$ (unión)
2. L_1L_2 (concatenación)
3. L_1^* (estrella de Kleene)
4. $L_1 \cap L_2$ (intersección)
5. $L_1 \setminus L_2$ (diferencia)

Cuatro formas de describir lo mismo



Cierre

Lo que vimos hoy

Parte	**La idea que hay que llevarse**
Lenguajes	Un texto es un programa de L solo si cumple la sintaxis y la semántica de L
Gramáticas	Generan las palabras de un lenguaje; la sintaxis abstracta quita los detalles
Autómatas	Reconocen las palabras; DFA y NFA reconocen los mismos lenguajes
Lenguajes regulares	Gramática regular = expresión regular = autómata finito; no saben contar

Repaso rápido (1 de 4)

¿Por qué [-7, 3, 14] no es una lista de la gramática de listas?

Repaso rápido (1 de 4)

¿Por qué [-7, 3, 14] no es una lista de la gramática de listas?

Porque el - y la coma no son terminales de la gramática, y los corchetes solo pueden ir vacíos: [].

Repaso rápido (2 de 4)

En el autómata producto, ¿qué estados son finales para la intersección?

Repaso rápido (2 de 4)

En el autómata producto, ¿qué estados son finales para la intersección?

Las parejas (p, q) donde p es final en el primero y q es final en el segundo.

Repaso rápido (3 de 4)

¿Hay lenguajes que un NFA reconoce y ningún DFA reconoce?

Repaso rápido (3 de 4)

¿Hay lenguajes que un NFA reconoce y ningún DFA reconoce?

No. Todo NFA tiene un DFA equivalente (construcción de subconjuntos).

Repaso rápido (4 de 4)

¿Por qué S ::= "a" S "b" | "ab" no es una gramática regular?

Repaso rápido (4 de 4)

¿Por qué S ::= "a" S "b" | "ab" no es una gramática regular?

Porque el no terminal queda entre dos terminales. Y su lenguaje, $a^n b^n$, no es regular (lema de bombeo).

Para comprobar

1. Escribe una expresión regular para las palabras sobre {a, b} que terminan en ab.
2. Dibuja un DFA para ese mismo lenguaje.
3. Alguien te dice que un NFA puede reconocer $a^n b^n$ porque puede adivinar. ¿Tiene razón?

Para la próxima

- Lecturas de la clase: Lee, cap. 2.1, 2.2, 2.11 a 2.13; Hopcroft, cap. 2, 3 y 4
- Practicar expresiones regulares: regexcrossword.com
- Los ejercicios del tema 01 de las notas
- Próxima tutoría: lenguajes libres de contexto

Apéndice

Más material de la clase de programming languages

Class work

Escribe una función que calcule el factorial de un número, en el lenguaje que quieras.

Class work

Escribe una función que calcule el factorial de un número, en el lenguaje que quieras.

¿Qué devuelve tu función con −3?

Las versiones de las notas

Scheme

```
(define (factorial n)
  (if (< n 2)
      1
      (* n (factorial (- n 1)))))
```

Rascal

```
int factorial(int n) =
  n < 2 ? 1 : n * factorial(n - 1);
```

Ada

```
function Factorial (N : Positive) return Positive is
  Result  : Positive := N;
  Counter : Natural  := N - 1;
begin
  for I in reverse 1..Counter loop
    Result := Result * I;
  end loop;
  return Result;
end Factorial;
```

CLU

```
factorial = proc (n: int) returns (int)
                          signals (negative)
  if n < 0 then signal negative
  elseif n = 0 then return(1)
  else return(n * factorial(n - 1))
  end
end factorial
```

Action!

```
CARD FUNC Factorial(INT n BYTE POINTER err)
  CARD i,res
  IF n<0 THEN
    err^=1 RETURN (0)
  ELSEIF n>8 THEN
    err^=2 RETURN (0)
  FI
  res=1
  FOR i=2 TO n
  DO
    res=res*i
  OD
  err^=0
RETURN (res)
```

Prolog

```
fact(X, 1) :- X < 2.
fact(X, F) :- Y is X-1, fact(Y, Z), F is Z*X.
```

Prolog

```
fact(X, 1) :- X < 2.
fact(X, F) :- Y is X-1, fact(Y, Z), F is Z*X.
```

```
?- fact(3, F).
F = 6 ;
F = 6 ;
F = 0 ;
F = 0 ;
F = 0 ...
```

Prolog, con guarda

```
fact(X, 1) :- X < 2.
fact(X, F) :- X >= 2, Y is X-1, fact(Y, Z), F is Z*X.
```

```
?- fact(3, F).
F = 6.

?- fact(-3, F).
F = 1.
```

¿Qué hace cada uno con n negativo?

Lenguaje	Con n negativo	Qué dice del diseño
Ada		
CLU		
Action!		
Prolog		

¿Qué hace cada uno con n negativo?

Lenguaje	Con n negativo	Qué dice del diseño
Ada	Error de tipos: el parámetro es Positive	El compilador atrapa el error
CLU	signal negative	El error es parte de la firma
Action!	Escribe 1 en err^ y devuelve 0	Sin excepciones: el error sale por un puntero
Prolog	Devuelve 1	La relación no excluye negativos

¿Qué cuenta como lenguaje?

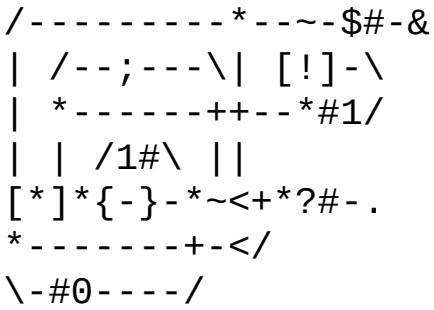
La clase también muestra lenguajes que no se parecen a nada de lo que has usado.

Scratch

- Lenguaje visual por bloques
- Pensado para niños y jóvenes de 8 a 16 años
- Los bloques solo encajan de formas válidas

ASCII Dots

- Lenguaje esotérico basado en arte ASCII
- Los puntos (.) viajan por caminos dibujados con caracteres
- Las operaciones transforman los puntos que pasan por ellas



0815

- Lenguaje esotérico basado en una cola
- Saltos y etiquetas como }:f: y ^:f:

```
}:r:        Start reader loop.
 |~         Read n,
 #:end:     if n is 0 terminates
 >=         enqueue it as the initial
            product, reposition.
 }:f:       Start factorial loop.
 x<:1:x-    Decrement n.
 {=*>       Dequeue product, position n,
            multiply, update product.
 ^:f:
 {+%        Dequeue incidental 0, add
            to get Y into Z, output fac(n).
 <:a:~$     Output a newline.
^:r:
```

Piet

- Lenguaje esotérico basado en una pila
- Los programas son imágenes que parecen pinturas abstractas
- 20 colores; los cambios de color son las instrucciones

SCuLPT

- Simple Cubic Language for Programming Tasks
- Los programas se construyen con piezas físicas en 3D
- Es Turing completo

UML

- Lenguaje visual de modelado de propósito general
- Diagramas de estructura, de comportamiento y de interacción

¿Qué cuenta como lenguaje?

Lenguaje	**Cómo se escribe un programa**
Scratch	Bloques que encajan; no se pueden escribir errores de sintaxis
ASCII Dots	Arte ASCII por donde viajan puntos
0815	Operaciones sobre una cola
Piet	Imágenes con 20 colores
SCuLPT	Piezas físicas en 3D; es Turing completo
UML	Diagramas de estructura, comportamiento e interacción

Ocho criterios (Pratt y Zelkowitz)

Criterio	**Ejemplo de hoy**
1. Claridad, simplicidad y unidad de conceptos	
2. Claridad de la estructura del programa	
3. Naturalidad para la aplicación	
4. Facilidad de extensión	
5. Soporte externo	
6. Facilidad de verificación	
7. Portabilidad	
8. Eficiencia y costo	

1. Claridad, simplicidad y unidad de conceptos

Un lenguaje es una herramienta para pensar, escribir y comunicar algoritmos. Debe tener pocos conceptos, reglas simples y regulares para combinarlos, y no debe ser ambiguo.

¿Qué lenguaje que conozcas tiene demasiados conceptos?

2. Claridad de la estructura del programa

La sintaxis no debe confundir, y la estructura del programa debe reflejar la estructura lógica del algoritmo.

¿Dónde la indentación de Python ayuda, y dónde la de C engaña?

3. Naturalidad para la aplicación

El lenguaje debe ofrecer las estructuras de datos, operaciones, estructuras de control y sintaxis que el problema necesita.

¿Para qué problema es natural SQL? ¿Para cuál no?

4. Facilidad de extensión

El lenguaje debe permitir extenderlo con mecanismos simples y naturales, construidos con sus propias primitivas.

¿Cómo se agrega un tipo nuevo en Python? ¿Se siente parte del lenguaje?

5. Soporte externo

Herramientas para probar, editar y guardar programas, y buena documentación. Deciden buena parte del éxito de un lenguaje.

¿Por qué escogiste el lenguaje que más usas?

6. Facilidad de verificación

Un programa se puede probar correcto con un método formal, validar con pruebas contra la especificación, o revisar leyéndolo. Una sintaxis y una semántica simples facilitan las tres.

¿Cuál de las tres formas usas tú más?

7. Portabilidad

Los programas deben poder moverse entre entornos de cómputo sin cambios.

¿Qué tan portable es un programa en C? ¿Y uno en Java?

8. Eficiencia y costo

Costo de ejecutar, de traducir (compilar), de crear (diseñar, programar, probar, modificar) y de mantener.

¿Cuál de los cuatro costos te importa más en un proyecto de clase?

Ocho criterios (Pratt y Zelkowitz)

Criterio	**Ejemplo de hoy**
1. Claridad, simplicidad y unidad de conceptos	Prolog lee = de una sola forma, pero no la que uno espera
2. Claridad de la estructura del programa	El árbol de arithmetic_logic refleja la estructura del texto
3. Naturalidad para la aplicación	Scratch para niños de 8 a 16 años
4. Facilidad de extensión	Los tipos abstractos definidos por el usuario en CLU
5. Soporte externo	Action! traía su propio IDE en el cartucho
6. Facilidad de verificación	Los subtipos Positive y Natural de Ada
7. Portabilidad	Action! compila a código del 6502 y solo corre en Atari
8. Eficiencia y costo	Ejecución, traducción, creación y mantenimiento

Tu turno

1. Escoge un lenguaje que conozcas: Python, Java, JavaScript, C, Haskell o Rascal
2. Escoge dos criterios
3. Di si el lenguaje cumple o falla cada uno, con un fragmento de código

JavaScript

```
[] + []
[] + {}
"5" - 2
"5" + 2
```

destroyallsoftware.com/talks/wat

JavaScript

```
[] + []   // ""
[] + {}   // "[object Object]"
"5" - 2   // 3
"5" + 2   // "52"
```

Principios de MacLennan (1 de 2)

Principio	**Qué dice**
Abstracción	No obligar a decir algo más de una vez; factorizar patrones
Automatización	Automatizar lo mecánico, tedioso o propenso a errores
Defensa en profundidad	Varias defensas: si una no atrapa el error, otra sí
Ocultamiento de información	El usuario de un módulo sabe lo necesario y nada más
Etiquetado	Nombres con significado en vez de posiciones en una lista
Costo localizado	Se paga solo por lo que se usa
Interfaz manifiesta	Las interfaces se ven en la sintaxis
Ortogonalidad	Funciones independientes, mecanismos independientes

Principios de MacLennan (2 de 2)

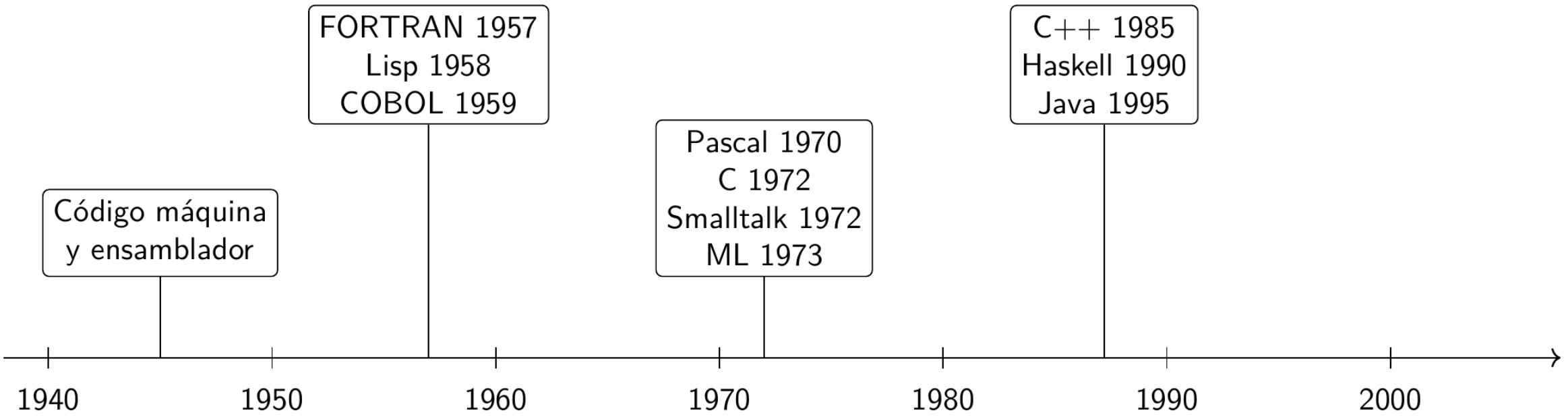
Principio	**Qué dice**
Portabilidad	Evitar lo que depende de una máquina en particular
Preservación de información	Dejar decir lo que el usuario sabe y el compilador necesita
Regularidad	Reglas sin excepciones
Seguridad	Ningún programa que viole la definición del lenguaje debe pasar sin detectarse
Simplicidad	Pocos conceptos con reglas simples de combinación
Estructura	La estructura estática refleja la estructura dinámica del cómputo
Consistencia sintáctica	Lo parecido se ve parecido; lo distinto se ve distinto
Cero, uno, infinito	Los únicos números razonables son 0, 1 e infinito

¿Qué principio se rompe?

Principio (MacLennan)	**Qué dice**
Regularidad	Reglas sin excepciones son más fáciles de aprender, usar e implementar
Consistencia sintáctica	Lo que se parece debe comportarse parecido; lo distinto debe verse distinto

Ortogonalidad y regularidad vuelven con gramáticas, ocultamiento de información con alcances, seguridad con sistemas de tipos.

De dónde vienen las ideas



Paradigmas

Paradigma	**Un programa es**	**Ejemplo**
Imperativo	Una secuencia de instrucciones que cambian el estado	C, Python
Funcional	Una expresión que se evalúa, sin estado mutable en el caso puro	Haskell, Scheme
Lógico	Hechos y reglas; ejecutarlo es hacer una consulta	Prolog
Orientado a objetos	Objetos que agrupan estado y comportamiento y se envían mensajes	Smalltalk, Java

Cuatro versiones de sum_evens

A

```
def sum_evens(xs):
    total = 0
    for x in xs:
        if x % 2 == 0:
            total += x
    return total
```

B

```
sumEvens :: [Int] -> Int
sumEvens = sum . filter even
```

C

```
SELECT SUM(value)
FROM numbers
WHERE value % 2 = 0;
```

D

```
sum_evens([], 0).
sum_evens([X|Xs], S) :-
    0 is X mod 2,
    sum_evens(Xs, S0),
    S is S0 + X.
sum_evens([X|Xs], S) :-
    1 is X mod 2,
    sum_evens(Xs, S).
```

Cuatro versiones de sum_evens

A. Imperativo (Python)

```
def sum_evens(xs):
    total = 0
    for x in xs:
        if x % 2 == 0:
            total += x
    return total
```

B. Funcional (Haskell)

```
sumEvens :: [Int] -> Int
sumEvens = sum . filter even
```

C. Declarativo (SQL)

```
SELECT SUM(value)
FROM numbers
WHERE value % 2 = 0;
```

D. Lógico (Prolog)

```
sum_evens([], 0).
sum_evens([X|Xs], S) :-
    0 is X mod 2,
    sum_evens(Xs, S0),
    S is S0 + X.
sum_evens([X|Xs], S) :-
    1 is X mod 2,
    sum_evens(Xs, S).
```

¿Y sin la guarda?

```
sum_evens([], 0).
sum_evens([X|Xs], S) :-
    0 is X mod 2,
    sum_evens(Xs, S0),
    S is S0 + X.
sum_evens([_|Xs], S) :-
    sum_evens(Xs, S).
```

```
?- sum_evens([1,2,3,4], S).
S = 6 ;
S = 2 ;
S = 4 ;
S = 0.
```

Cuatro tensiones de diseño

Tensión	**Ejemplo de hoy**
Expresividad y simplicidad	
Seguridad y flexibilidad	
Rendimiento y abstracción	
Legibilidad y facilidad de escritura	

Cuatro tensiones de diseño

Tensión	**Ejemplo de hoy**
Expresividad y simplicidad	B cabe en una línea, pero hay que saber composición de funciones
Seguridad y flexibilidad	Haskell rechaza a = b + c antes de correr; Python la corre y falla
Rendimiento y abstracción	Una lista de Python guarda punteros a objetos; un arreglo de int en C es un bloque contiguo
Legibilidad y facilidad de escritura	0815 y Piet son cortos de escribir e ilegibles para casi todos